

AWS Mastery: 26 AWS Cloud Projects for Engineers & Architects

Solution Document

© Pravin Mishra, The CloudAdvisory, or its affiliates. All rights reserved. This work may not be reproduced or redistributed, in whole or in part, without prior written permission from Pravin Mishra. Commercial copying, lending, or selling is prohibited. All trademarks are the property of their owners.

Module Overview:

This Project follows the **AWS Compute Services** module of the Udemmy course **Cloud Computing with AWS for Beginners**. It is designed to provide you with the foundational knowledge and skills required to start using Amazon Compute Services. This module covers a range of introductory topics, including:

- **Compute:** Learning how computers operate and complete tasks within the cloud setup.
- **Compute Services:** Discovering different cloud services that enable computing for various needs.
- **Virtual Machine:** An overview of virtual machines, Simulated computers that operate within a real computer system, performing tasks similarly to actual computers.
- **Elastic Compute Cloud (EC2):** Details about a service called EC2 that offers flexible and resizable computing power on the internet.
- **Component of Amazon EC2:** Different parts or elements that make up the Amazon EC2 service, each contributing to its functionalities and configurations.
- **EC2 Instance Naming Convention:** Guidelines or rules used to name and identify specific instances created within the EC2 service.
- **Instance Types:** Different categories or variations of virtual machines available on AWS, each with its own specific configurations and capabilities to suit diverse computing requirements and workloads.

- **Launch Your First EC2 Instance:** Step-by-step guidance on starting your initial virtual server within the AWS cloud environment.
- **SSH EC2 Instance and Install Httpd:** Guidance on securely accessing an EC2 instance using SSH and setting up an HTTP server (Httpd) on it.
- **Creating a Windows EC2 Instance:** The process of setting up a virtual Windows-based computer within the AWS cloud environment.
- **Elastic Container Service (ECS):** An explanation of AWS ECS service, that is used for efficiently managing and deploying containers, simplifying application development and scaling in the cloud environment.
- **AWS Elastic Beanstalk:** A look into AWS Elastic Beanstalk, a platform that streamlines the deployment and management of applications in the cloud by automating the deployment process, making it easier to handle application-related tasks without the need for manual configuration.
- **Serverless Computing:** Understanding of Serverless Computing, where cloud providers manage the infrastructure and resources, allowing developers to focus solely on writing and deploying code without the need to manage servers or infrastructure configurations.
- **AWS Lambda:** A look into AWS Lambda serverless computing service that enables users to run code without provisioning or managing servers.

Project Objectives:

After completing this project, you should be able to do the following:

- Launch and configure a Linux-based EC2 instance precisely customized to fulfill the operational needs of EpicReads, an organization operating in the cloud environment.
- To familiarize yourself with Virtual Machines, Elastic Compute Cloud (EC2) services, Elastic Container Service (ECS), and explore the fundamentals of serverless computing for a comprehensive understanding.

DURATION

This project requires approximately 90 minutes to complete.

ICON KEY

Throughout this Project, we use various icons to highlight different types of instructions and notes. Here's what each icon signifies:

-  **Note:** Offers a helpful hint, tip, or piece of crucial guidance.
-  **Learn More:** Directs you to additional sources for more detailed information.
-  **Caution:** Points out information that, while not critical, is important and missing it might necessitate redoing steps.
-  **WARNING:** Indicates an action that's permanent with potential consequences that could cause process failure or irreversible changes.
-  **Expected Output:** Shows a sample of the expected result to help confirm correct execution of a command or file edit.
-  **Command:** Denotes a specific command that you're required to execute.
-  **Consider:** Encourages you to reflect on how to apply a concept in your scenario or to spark a discussion related to the topic at hand.

COMMON ERRORS

1. **Misconfiguration of Security Groups:** Not setting up appropriate inbound and outbound rules in security groups can lead to connectivity issues or security vulnerabilities.
2. **Improper Setup of SSH and HTTPd:** Incorrectly configuring SSH access or setting up an HTTP server (Httpd) on an EC2 instance can result in security gaps or functionality issues.

Refer to [COMMON ERRORS - EXPLANATION SECTION](#) for the solution.

Start Project: **Welcome to Our AWS Compute Solutions Page!**

My Greetings and a warm welcome to our Second AWS Project Solutions Page. Your journey into harnessing AWS compute power begins here, and I am happy to be a part of this exciting journey.

Let's start on this Compute journey together!

In this solution page, you will walk through the step-by-step process of the tasks:

1. Launch a Linux-based EC2 instance using Amazon Linux 2 AMI.
2. Configure the instance with the necessary settings for web server operations.
3. Establish a secure SSH connection and perform basic network tests.

Let's start now!

Task 1: Setting Up an AWS Virtual Private Cloud (VPC) with Public and Private Subnets for EpicReads

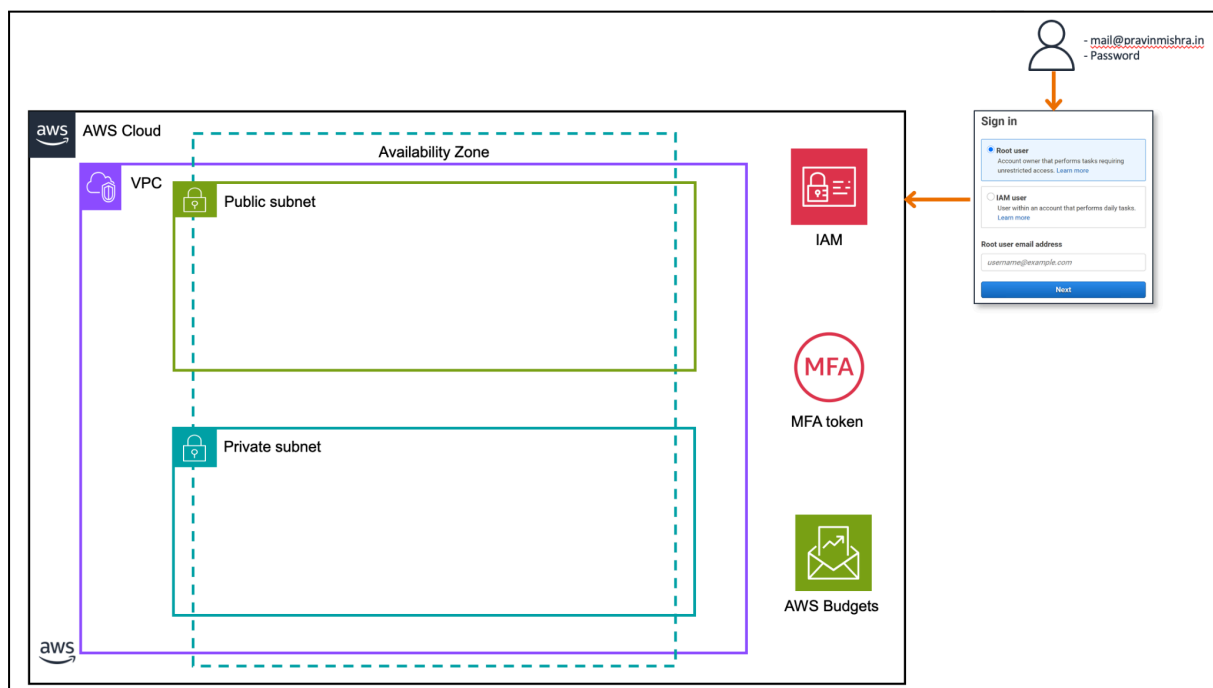


Figure 1.1.1: Virtual Private Cloud

Project Overview:

This project focuses on creating a VPC within the AWS environment for EpicReads, complete with public and private subnets. This infrastructure will provide a secure and isolated environment for hosting the company's applications and services.

Project Objectives:

- Establish a VPC that offers enhanced security and network isolation for EpicReads' AWS resources.
- Create public subnets for resources that need internet access and private subnets for resources that require restricted access.
- Ensure that the VPC design aligns with best practices for security and scalability.

STEP 1: Planning and Design:

Determine the network range (CIDR block) for the VPC and subnets.

1. Understand CIDR Notation Basics:

- CIDR notation represents an IP address range using a combination of an IP address and a suffix (e.g., /16, /24) denoting the number of bits in the network prefix.
- A smaller suffix represents a larger number of available IP addresses for hosts.

2. List Required Subnets and Their Sizes:

- Plan the number of subnets you need and their respective sizes (number of IP addresses).
- Separate these subnets into public and private based on their need for internet accessibility.
- When deciding the IP address range for your AWS Virtual Private Cloud (VPC), you can use tools like a CIDR calculator available online or in Google to help set up the network. For instance, we chose a CIDR block of 10.0.0.0/16 for our VPC.
- Within this VPC, the plan is to create subnets that are like smaller segments within the larger network. For example, we aim to make two public subnets (e.g., 10.0.0.0/24, 10.0.1.0/24) and two private subnets (e.g., 10.0.2.0/24, 10.0.3.0/24). (*Figure 1.1.2: CIDR block selection*)
- The '10.0.0.0/16' CIDR block allows us a range of IP addresses for all our subnets. Each subnet gets its own portion of this range, defining the number of available IP addresses for devices or instances within that subnet.

Network Address Block	Subnet Mask	No. of Hosts/Subnet	Number of Subnets
10.0.0.0/16	255.255.255.0/24	256	256
Host Address Range	Broadcast Address	Wildcard Mask	CIDR Notation
10.0.0.1 - 10.0.0.254	10.0.0.255	0.0.0.255	10.0.0.0/24

Subnet Details

Subnet ID	Subnet Address	Host Address Range	Broadcast Address
1	10.0.0.0	10.0.0.1 - 10.0.0.254	10.0.0.255
2	10.0.1.0	10.0.1.1 - 10.0.1.254	10.0.1.255
3	10.0.2.0	10.0.2.1 - 10.0.2.254	10.0.2.255
4	10.0.3.0	10.0.3.1 - 10.0.3.254	10.0.3.255

Figure 1.1.2: CIDR block selection

⚠ Caution: Incorrect CIDR block selection or subnet planning may lead to IP address conflicts, network connectivity issues, or security vulnerabilities. Pay close attention to details to avoid complications.

STEP 2: Creating the VPC:

1. Access the AWS Console:

- Log in to your AWS account and navigate to the AWS Management Console.

2. Choose the Desired Region:

- At the topmost corner of the AWS Management Console, select the region where you want to create your VPC. For instance, if you're selecting the Asia Pacific region, choose the appropriate region from the dropdown menu (*Figure 1.1.3: Region Selection*)

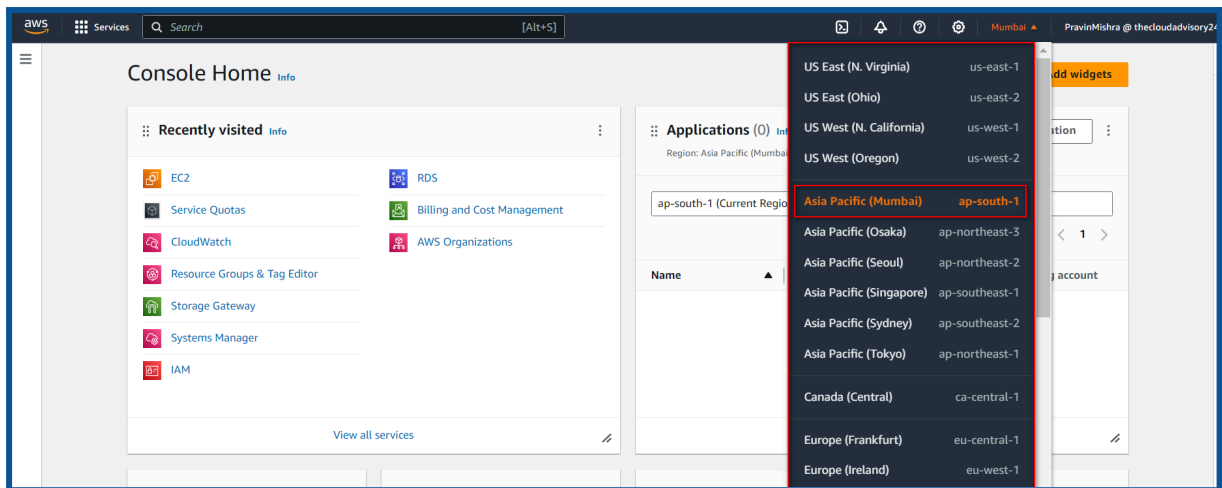


Figure 1.1.3: Region Selection

3. Go to the VPC Dashboard:

- In the AWS Management Console, locate the "Services" menu and select "VPC" under the "Networking & Content Delivery" section.

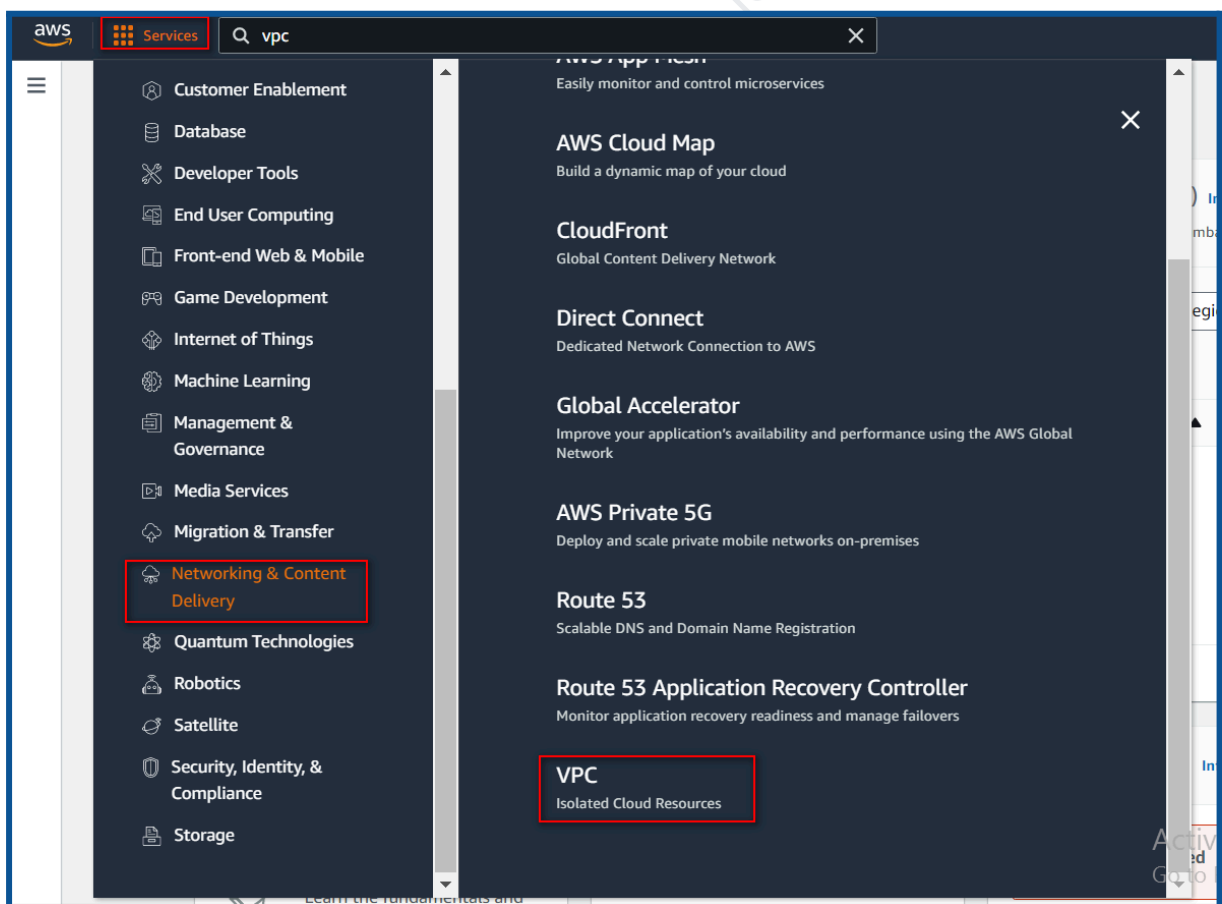


Figure 1.1.4: Select VPC

4. Create a New VPC:

- Click on "Create VPC" in the left sidebar of VPC Dashboard.
(Figure 1.1.5: Create VPC)

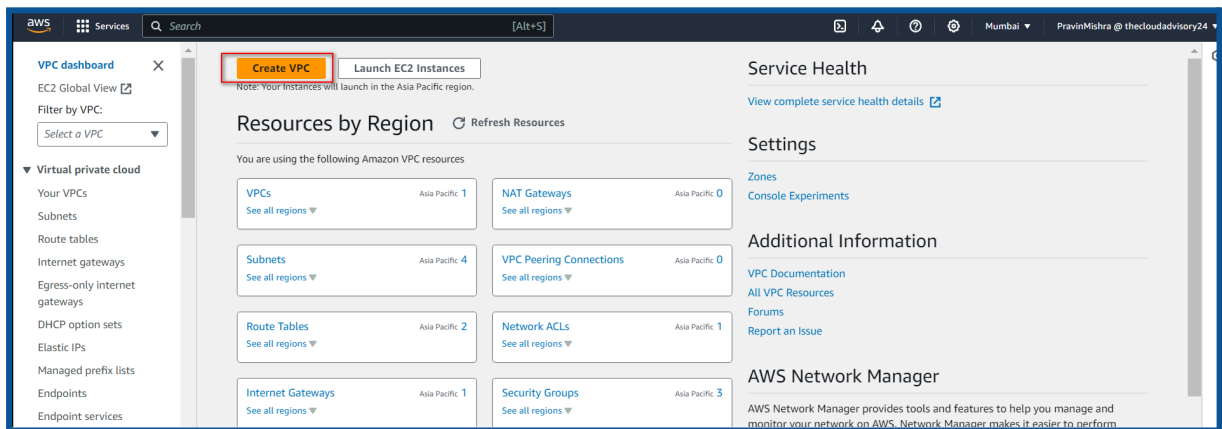


Figure 1.1.5: Create VPC

- In the resource to create option click on "**VPC only**" and provide the name of the VPC in the name tag.
- Enter the IPv4 CIDR block you previously selected in the earlier steps (for example, **10.0.0.0/16**) in the "**IPv4 CIDR block**" field. This defines the range of private IP addresses for your VPC's instances. (Figure 1.1.6: VPC Settings)

aws Services Search [Alt+S]

VPC > Your VPCs > Create VPC

Create VPC [Info](#)

A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.

VPC settings

Resources to create [Info](#)
Create only the VPC resource or the VPC and other networking resources.

☒ VPC only ☐ VPC and more

Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.

VPC-EpicReads

IPv4 CIDR block [Info](#)

☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR

10.0.0.0/16

CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)

☒ No IPv6 CIDR block
☐ IPAM-allocated IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

Figure 1.1.6: VPC Settings

- Select "**Default**" for the tenancy option. This indicates that your instances will be provisioned on shared hardware.
- Tags are optional but helpful for identification purposes. You can add tags to your VPC by clicking "Add tag" and providing a key-value pair. (Figure 1.1.7: VPC Tenancy)

aws Services Search [Alt+S]

IPv4 CIDR block [Info](#)

☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR

CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)

☒ No IPv6 CIDR block
☐ IPAM-allocated IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

Tenancy [Info](#)

Tags
 A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Name	VPC-EpicReads	Remove tag

You can add 49 more tags

Figure 1.1.7: VPC Tenancy

- Review the details you've entered for the VPC.
- Once everything looks correct, click on **"Create VPC"** to initiate the creation process.
- VPC now creates and views the details in the VPC Dashboard.
 (Figure 1.1.8: VPC Details)

aws Services Search [Alt+S] Mumbai PravinMishra @ thecloudadvisory24

VPC dashboard X
 EC2 Global View
 Filter by VPC:

▼ Virtual private cloud

▼ Your VPCs

Subnets
 Route tables
 Internet gateways
 Egress-only internet gateways
 DHCP option sets
 Elastic IPs
 Managed prefix lists
 Endpoints
 Endpoint services
 NAT gateways
 Peering connections

▼ Security

Network ACLs

VPC > Your VPCs > vpc-08cfcea0339536856

vpc-08cfcea0339536856 / VPC-EpicReads

Details [Info](#)

VPC ID vpc-08cfcea0339536856	State ✔ Available	DNS hostnames Disabled	DNS resolution Enabled
Tenancy Default	DHCP option set dopt-0c5180790e8d97f5c	Main route table rtb-02bf36dbcdca96b5	Main network ACL acl-076da6f54c455e594
Default VPC No	IPv4 CIDR 10.0.0.0/16	IPv6 pool -	IPv6 CIDR (Network border group) -
Network Address Usage metrics Disabled	Route 53 Resolver DNS Firewall rule groups -	Owner ID 052260988726	

Resource map [New](#) | CIDRs | Flow logs | **Tags** | Integrations

Tags

Key	Value
Name	VPC-EpicReads

< 1 >

Figure 1.1.8: VPC Details

Note: While creating your VPC and configuring subnets, always consider the specific requirements of your applications and services.

STEP 3: Setting Up Subnets:

1. Create Public Subnets:

- In the VPC Dashboard, navigate to "Subnets" on the left sidebar and click on "Create subnet." (Figure 1.1.9: Create Subnet)

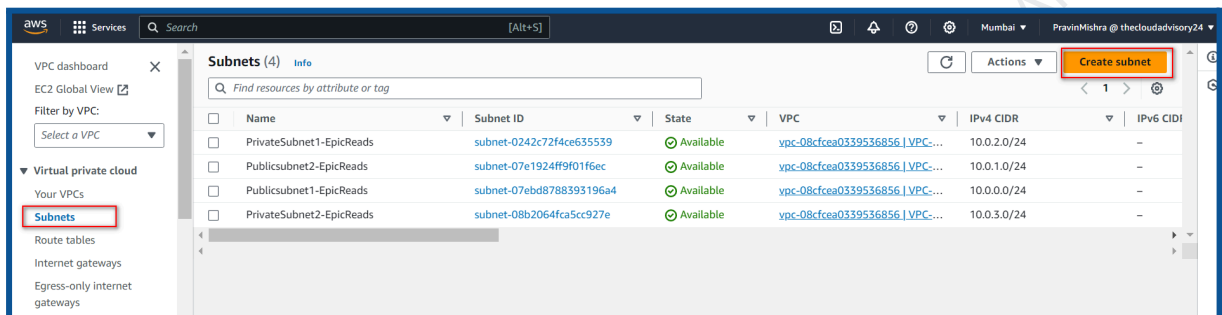


Figure 1.1.9: Create Subnet

- Choose the VPC you created earlier from the dropdown menu. (Figure 1.1.10: Subnet settings)

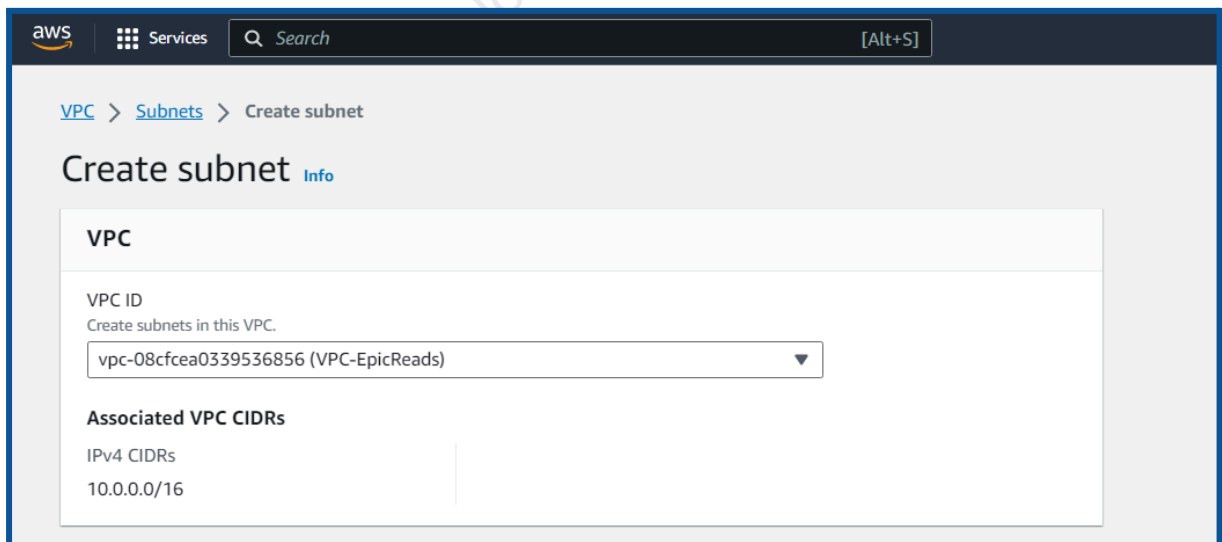


Figure 1.1.10: Subnet settings

- Provide a name for the subnet in the "Name tag" field to identify it easily.
- Assign an appropriate CIDR block for the public subnet (for example, 10.0.1.0/24).

- Ensure to select a different Availability Zone for each subnet if you plan to have multiple subnets for high availability. (*Figure 1.1.11: Subnet name*)

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

IPv4 VPC CIDR block [Info](#)
Choose the IPv4 VPC CIDR block to create a subnet in.

IPv4 subnet CIDR block
 256 IPs

Tags - optional

Key	Value - optional	
Name	PublicSubnet1-EpicReads	Remove
Add new tag		
You can add 49 more tags.		
Remove		
Add new subnet		

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

PublicSubnet1-EpicReads

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

Asia Pacific (Mumbai) / ap-south-1a

IPv4 VPC CIDR block [Info](#)
Choose the IPv4 VPC CIDR block to create a subnet in.

10.0.0.0/16

IPv4 subnet CIDR block

10.0.1.0/24 256 IPs

< > ^ v

▼ Tags - optional

Key	Value - optional	
Q Name	Q PublicSubnet1-EpicReads	Remove

Add new tag

You can add 49 more tags.

Remove

Add new subnet

Cancel Create subnet

Figure 1.1.11: Subnet name

Repeat the same for *PublicSubnet2-EpicReads* with CIDR block *10.0.1.0/24*

2. Create Private Subnets:

- Similarly, click on "Create subnet" again within the VPC Dashboard.
- Provide a name for the private subnet in the "Name tag" field.
- Choose the same VPC as before from the dropdown menu.
- Assign a different CIDR block for the private subnet (for example, 10.0.2.0/24).
- Remember to select different Availability Zones for the private subnets to ensure distribution across multiple zones. (*Figure 1.1.12: Subnet configuration*)

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.
PrivateSubnet1-EpicReads
The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.
Asia Pacific (Mumbai) / ap-south-1b

IPv4 VPC CIDR block [Info](#)
Choose the IPv4 VPC CIDR block to create a subnet in.
10.0.0.0/16

IPv4 subnet CIDR block
10.0.2.0/24 256 IPs
< > ^ v

Tags - optional

Key	Value - optional	
Name	PrivateSubnet1-EpicReads	Remove

[Add new tag](#)
You can add 49 more tags.

[Remove](#)

[Add new subnet](#)

[Cancel](#) [Create subnet](#)

Figure 1.1.12: Subnet configuration

Repeat the same for *PrivateSubnet2-EpicReads* with *CIDR block 10.0.3.0/24*

4. Review and Create:

- Double-check the details you've entered for the subnets.
- Ensure that each subnet has its unique CIDR block and is associated with the appropriate VPC and Availability Zone.
- Click on "Create subnet" to complete the creation of both public and private subnets.

Get back to the VPC console, Select the Subnets and you will find the Subnets that you have created a while ago! (Figure 1.1.13: Subnets of EpicReads)

VPC dashboard
EC2 Global View

Filter by VPC:
Select a VPC

Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs
Managed prefix lists
Endpoints
Endpoint services
NAT gateways

Subnets (18) Info

Find resources by attribute or tag

Name	Subnet ID	State	VPC	IPv4 CIDR	IPv6 CIDR	Available
-	subnet-0c7a61b938...	Available	vpc-062704e76632501b3	172.31.16.0/20	-	4091
-	subnet-0171a9c49d...	Available	vpc-062704e76632501b3	172.31.32.0/20	-	4091
-	subnet-0d4771fc16c...	Available	vpc-062704e76632501b3	172.31.0.0/20	-	4091
PrivateSubnet1-EpicReads	subnet-04ec8d94f50...	Available	vpc-0c5d4c7976a4c205a VPC...	10.0.2.0/24	-	251
PrivateSubnet2-EpicReads	subnet-08a34ee00b...	Available	vpc-0c5d4c7976a4c205a VPC...	10.0.4.0/24	-	251
PublicSubnet1-EpicReads	subnet-0070c58c8e...	Available	vpc-0c5d4c7976a4c205a VPC...	10.0.1.0/24	-	250
PublicSubnet2-EpicReads	subnet-00130d1a60...	Available	vpc-0c5d4c7976a4c205a VPC...	10.0.3.0/24	-	250
RDS-Pvt-subnet-1	subnet-0840984634...	Available	vpc-06ac8873143b92107 WP...	10.15.4.0/25	-	123
RDS-Pvt-subnet-2	subnet-09a15af802...	Available	vpc-06ac8873143b92107 WP...	10.15.4.128/25	-	123
RDS-Pvt-subnet-3	subnet-028d1d89c6...	Available	vpc-06ac8873143b92107 WP...	10.15.5.0/25	-	123
WP-PrivateSubnet1	subnet-026dacd685...	Available	vpc-06ac8873143b92107 WP...	10.15.2.0/24	-	251
WP-PrivateSubnet1	subnet-065d295f7d...	Available	vpc-0358d14252cf5643b WP...	10.0.1.0/24	-	251
WP-PrivateSubnet2	subnet-05ed6162c2...	Available	vpc-06ac8873143b92107 WP...	10.15.3.0/24	-	251
WP-PrivateSubnet2	subnet-0a4c9a4e0ff...	Available	vpc-0358d14252cf5643b WP...	10.0.2.0/24	-	251

Figure 1.1.13: Subnets of EpicReads

Consider: Reflect on the specific requirements of your applications and services when planning subnets. Consider future scalability and potential changes in network demands.

STEP 4: Configuring Internet Access:

1. Set Up Internet Gateway (IGW) for Public Subnets:

- In the VPC Dashboard, click on "Internet Gateways" in the left sidebar. (Figure 1.1.14: Create Internet Gateway)

VPC dashboard
EC2 Global View

Filter by VPC:
Select a VPC

Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs

Internet gateways (1) Info

Search

Name	Internet gateway ID	State	VPC ID	Owner
Internetgateway-EpicReads	igw-02780d863b863d342	Attached	vpc-08cfcea0339536856 VPC-EpicReads	052260988726

Create internet gateway

Figure 1.1.14: Create Internet Gateway

- Click on "Create internet gateway" and provide a name for the IGW. (Figure 1.1.15: Internet Gateway Settings)

Create internet gateway [Info](#)

An internet gateway is a virtual router that connects a VPC to the internet. To create a new internet gateway specify the name for the gateway below.

Internet gateway settings

Name tag
Creates a tag with a key of 'Name' and a value that you specify.

internetgateway-EpicReads

Tags - optional
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Name	internetgateway-EpicReads	Remove

Add new tag

You can add 49 more tags.

Cancel **Create internet gateway**

Figure 1.1.15: Internet Gateway Settings

- Select the newly created IGW and click on "Attach to VPC," then choose your VPC from the list. (Figure 1.1.16: Attach to VPC)

The following internet gateway was created: igw-074c956117bee2f60 - internetgateway-EpicReads. You can now attach to a VPC to enable the VPC to communicate with the internet.

Attach to a VPC

VPC > Internet gateways > igw-074c956117bee2f60

igw-074c956117bee2f60 / internetgateway-EpicReads

Details [Info](#)

Internet gateway ID	State	VPC ID	Owner
igw-074c956117bee2f60	Detached	-	052260988726

Tags

Key	Value
Name	internetgateway-EpicReads

Manage tags

Figure 1.1.16: Attach to VPC

- Once VPC is attached, you can see the details in the internet gateways Tab. (Figure 1.1.17: IGW Details)

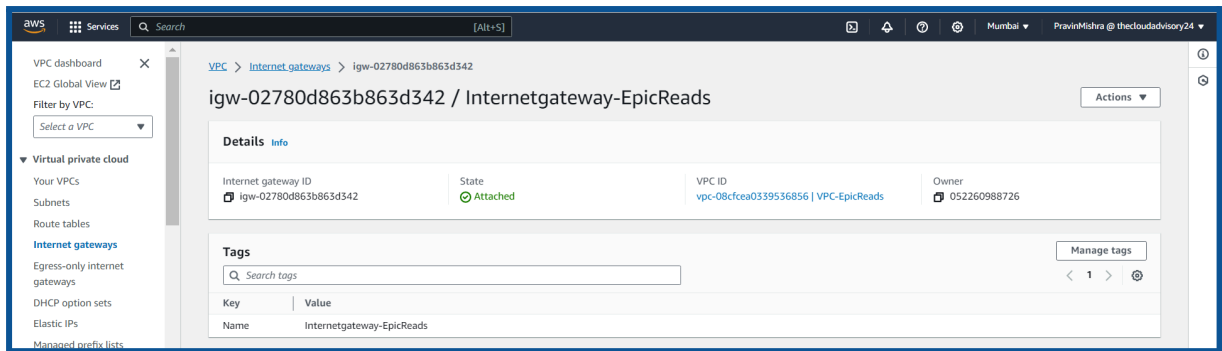


Figure 1.1.17: IGW Details

2. Update Route Tables for Public Subnets:

- Go back to the VPC Dashboard and click on "Route Tables" in the left sidebar. (Figure 1.1.18: Route Tables)

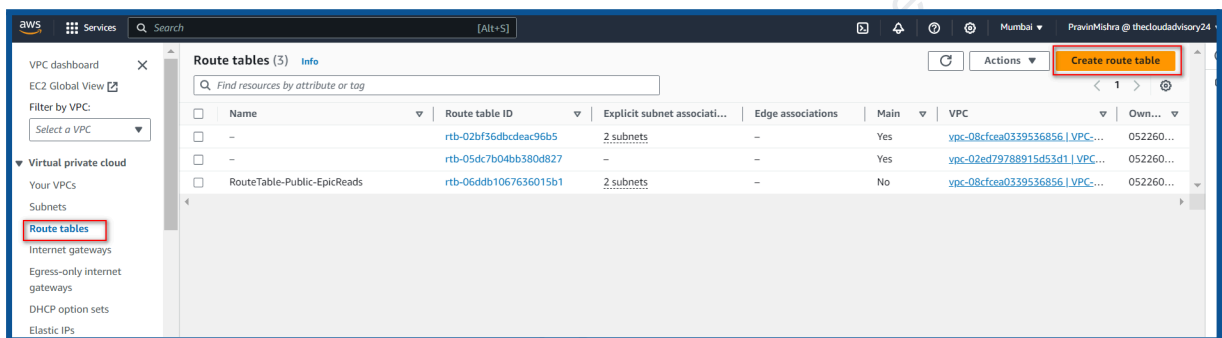


Figure 1.1.18: Route Tables

- Select the route table associated with your public subnets (usually the main route table for the VPC) (Figure 1.1.19: Route Table Settings).

aws Services Search [Alt+S]

VPC > Route tables > Create route table

Create route table [Info](#)

A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

RouteTable-Public-EpicReads

VPC
The VPC to use for this route table.

vpc-02ed79788915d53d1 (VPC-EpicReads)

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key **Value - optional**

Name X RouteTable-Public-EpicReads X Remove

Add new tag

You can add 49 more tags.

Cancel **Create route table**

Figure 1.1.19: Route Table Settings

- The Route Table is created now.
- Click on the "**Routes**" tab and then on "**Edit routes.**"
- Add a new route with destination **0.0.0.0/0** and target as the ID of the IGW you attached earlier. (Figure 1.1.20: Edit Routes)

aws Services Search [Alt+S] Mumbai PravinMishra @ thecloudadvisory24

VPC > Route tables > rtb-06ddb1067636015b1 > Edit routes

Edit routes

Destination	Target	Status	Propagated
10.0.0.0/16	local	Active	No
0.0.0.0/0	Internet Gateway	Active	No

Add route

Cancel Preview **Save changes**

Figure 1.1.20: Edit Routes

- Save the changes.

3. Associate Public Subnets to the Routetable

- As the Public Subnets need the internet access do associate *PublicSubnet1-EpicReads* and *PublicSubnet2-EpicReads* to the created Route table *Routetable-Public-EpicReads*
- Select the Route table *Routetable-Public-EpicReads*, Choose *Subnet Associations* and Click on *Edit subnet association* (Figure 1.1.21: Subnet Association)

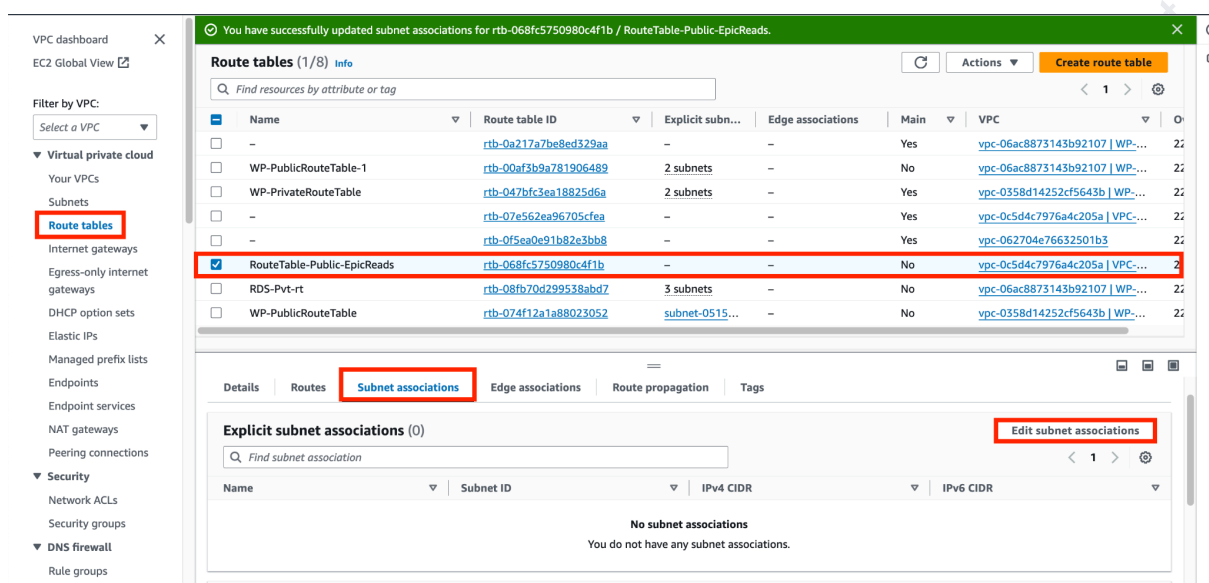


Figure 1.1.21: Subnet Association

Select both the *Public subnets* of EpicReads and click on the button *Save association* (Figure 1.1.22: Choose Public Subnets)

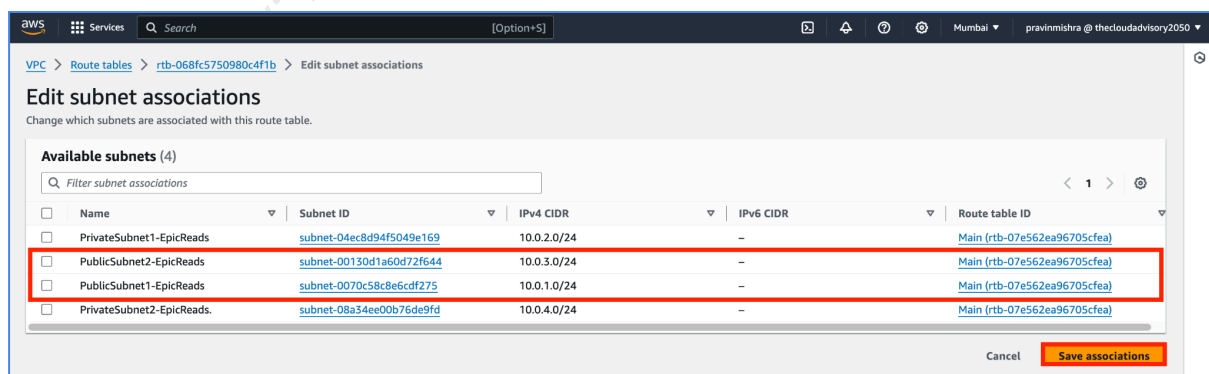


Figure 1.1.22: Choose Public Subnets

4. Review and Confirm:

- Double-check that the IGW is attached to the VPC and the route table for public subnets has the necessary route to the IGW.
- Confirm that the route table is associated with the public subnets you want to have internet access.

STEP 5: Implementing Network Security:

1. Create Network Access Control Lists (NACLs):

- In the VPC Dashboard, click on "Network ACLs" in the left sidebar.
- Click on "Create network ACL." (Figure 1.1.23: Create Network ACL)

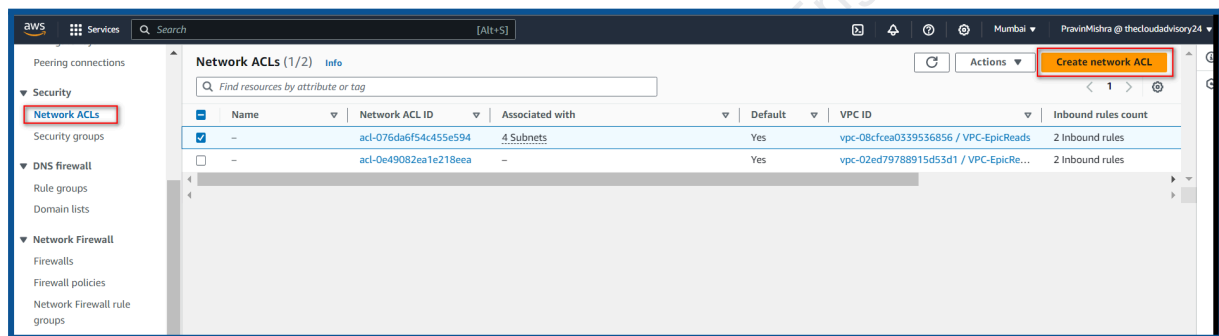


Figure 1.1.23: Create Network ACL

- Provide a name for the NACL and associate it with your VPC. (Figure 1.1.24: Network ACL Settings)

aws Services Search [Alt+S]

VPC > Network ACLs > Create network ACL

Create network ACL [Info](#)

A network ACL is an optional layer of security that acts as a firewall for controlling traffic in and out of a subnet.

Network ACL settings

Name - optional
Creates a tag with a key of 'Name' and a value that you specify.

NACL_EpicReads

VPC
VPC to use for this network ACL.

vpc-02ed79788915d53d1 (VPC-EpicReads)

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Name	NACL_EpicReads	Remove tag

Add tag

You can add 49 more tags

Cancel Create network ACL

Figure 1.1.24: Network ACL Settings

- Configure inbound and outbound rules to control traffic.
- Remember that NACLs are stateless, so you'll need to define rules for both inbound and outbound traffic. (Figure 1.1.25: Edit Inbound Rules)

aws Services Search [Alt+S]

VPC > Network ACLs > acl-076da6f54c455e594 > Edit inbound rules

Edit inbound rules [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the VPC.

Rule number	Type	Protocol	Port range	Source	Allow/Deny	
100	All traffic	All	All	0.0.0.0/0	Allow	Remove
*	All traffic	All	All	0.0.0.0/0	Deny	

Add new rule Sort by rule number

Cancel Preview changes Save changes

Figure 1.1.25: Edit Inbound Rules

2. Create Security Groups:

- Go back to the VPC Dashboard and click on "Security Groups" in the left sidebar.
- Click on "Create security group." (Figure 1.1.26: Create security group)

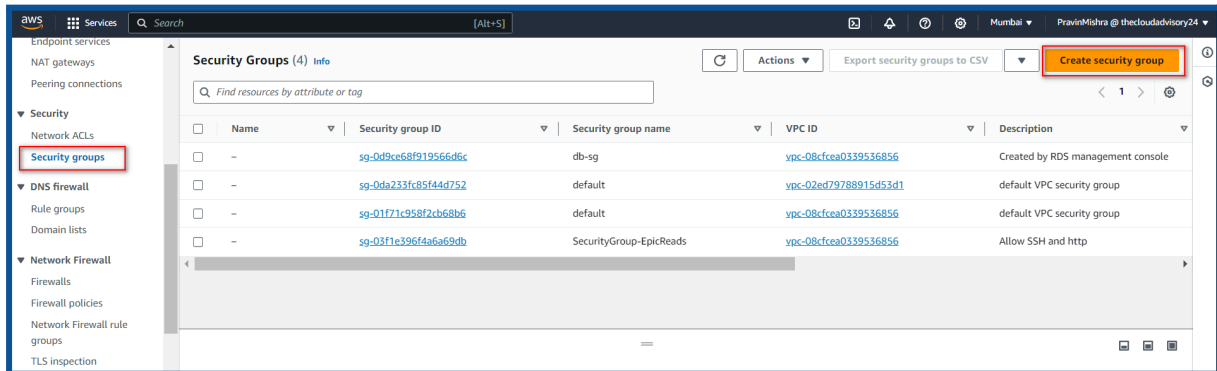


Figure 1.1.26: Create security group

- Create security group
- Provide a name and description for the security group. (Figure 1.1.27: security group Details)

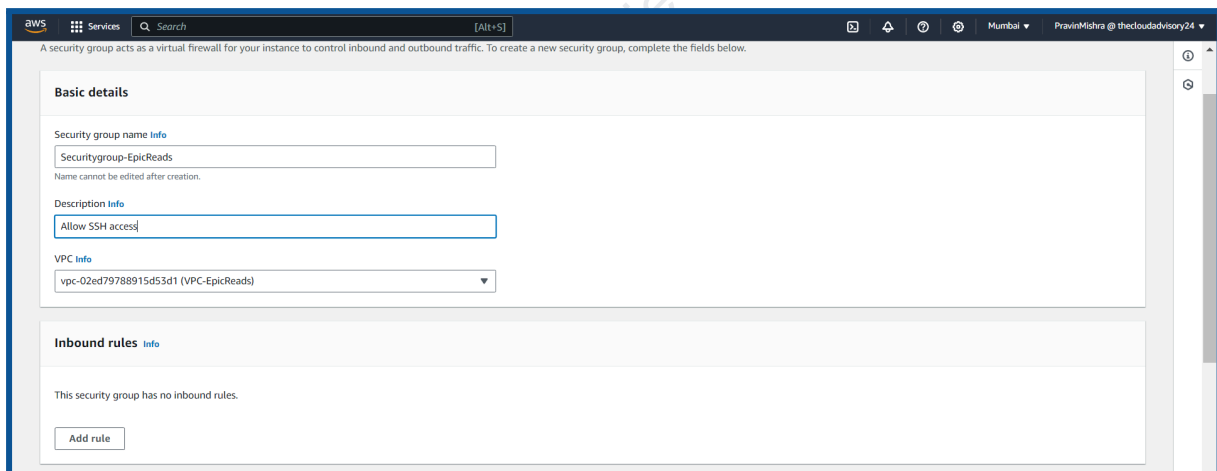


Figure 1.1.27: security group Details

- Define or Edit inbound and outbound rules for the security group, specifying the allowed traffic types and sources/destinations. (Figure 1.1.28: security group inbound)

(Here in the below inbound rules Mysql/ Aurora can be avoided and can be shown in the *Assignment 2: Enhancing Security with Security Groups for EpicReads*

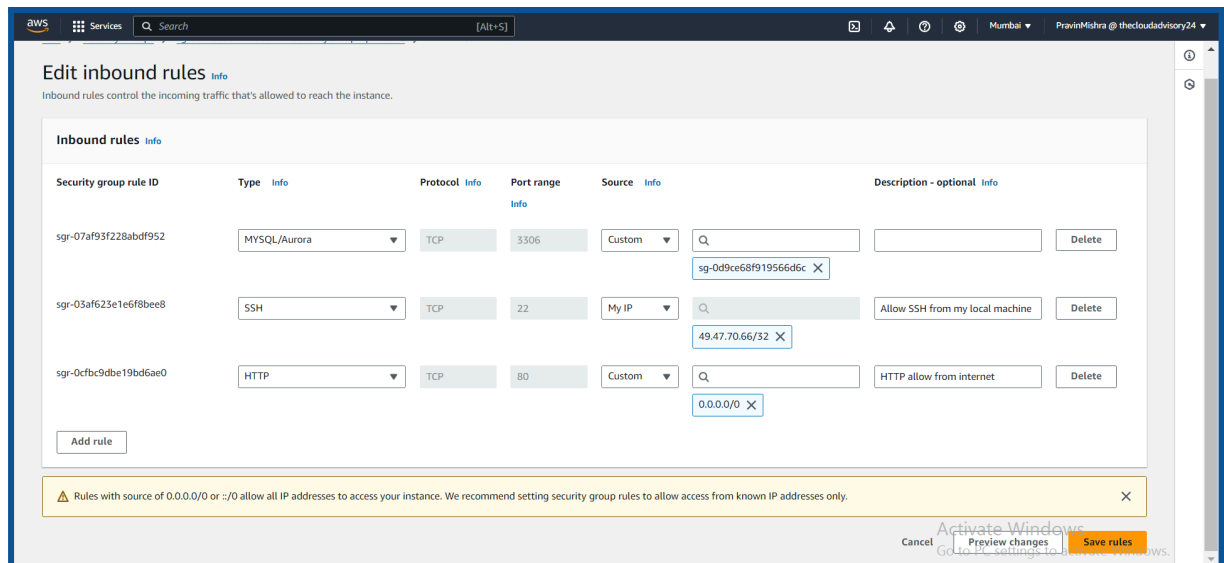


Figure 1.1.28: security group inbound

3. Apply NACLs and Security Groups to Subnets:

- Associate the NACLs and Security Groups with the appropriate subnets.
- In the VPC Dashboard, click on "Subnets" in the left sidebar.
- Select a subnet, go to the "Network ACL" tab, and associate the relevant NACL.
- Similarly, select a subnet, go to the "Security" tab, and associate the relevant Security Group. (Figure 1.1.26: security group Details) (This step and the Figure not much clear)

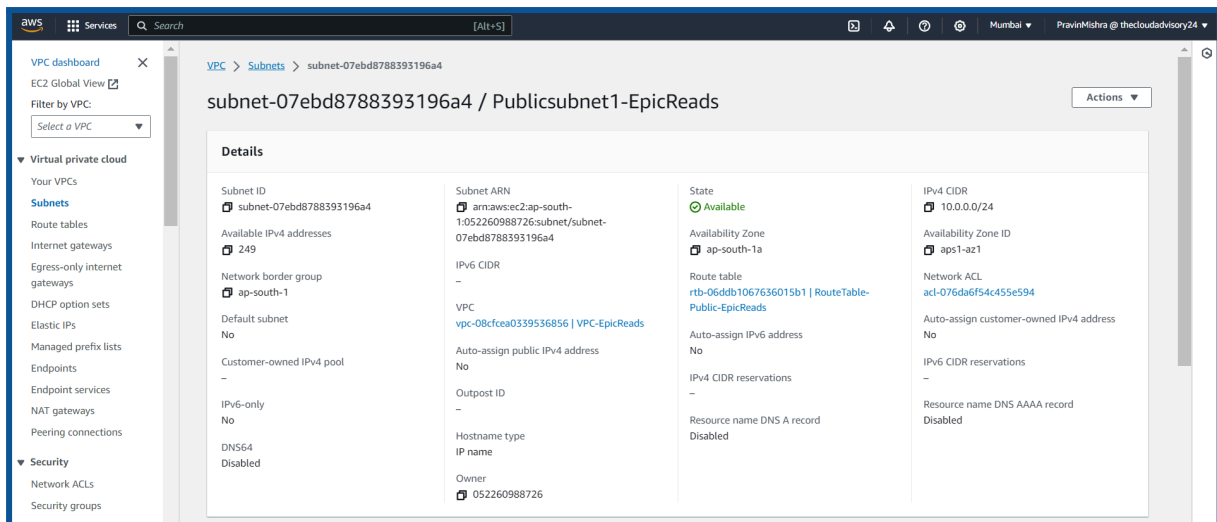


Figure 1.1.26: security group Details

4. Security Best Practices for Private Subnets:

- For private subnets, ensure that the associated NACL and Security Group configurations restrict inbound traffic to only what is necessary.
- Specifically, deny inbound traffic that is not required for the internal functioning of your resources.
- This helps ensure that resources in private subnets are not directly accessible from the internet.

5. Review and Confirm:

- Double-check the configurations of NACLs and Security Groups for both public and private subnets.

✓ **Expected Output:** After completing this task, you should have a VPC with properly configured public and private subnets, an attached Internet Gateway (IGW), and updated route tables. The network should follow best practices, providing secure and scalable infrastructure.

Task 2. Launching and Configuring a Linux EC2 Instance at EpicReads

STEP 1: Launching an EC2 Instance in AWS

- a. **Sign in to AWS Console:** Go to the AWS Management Console at <https://console.aws.amazon.com/> (Figure 1.1: Search result).

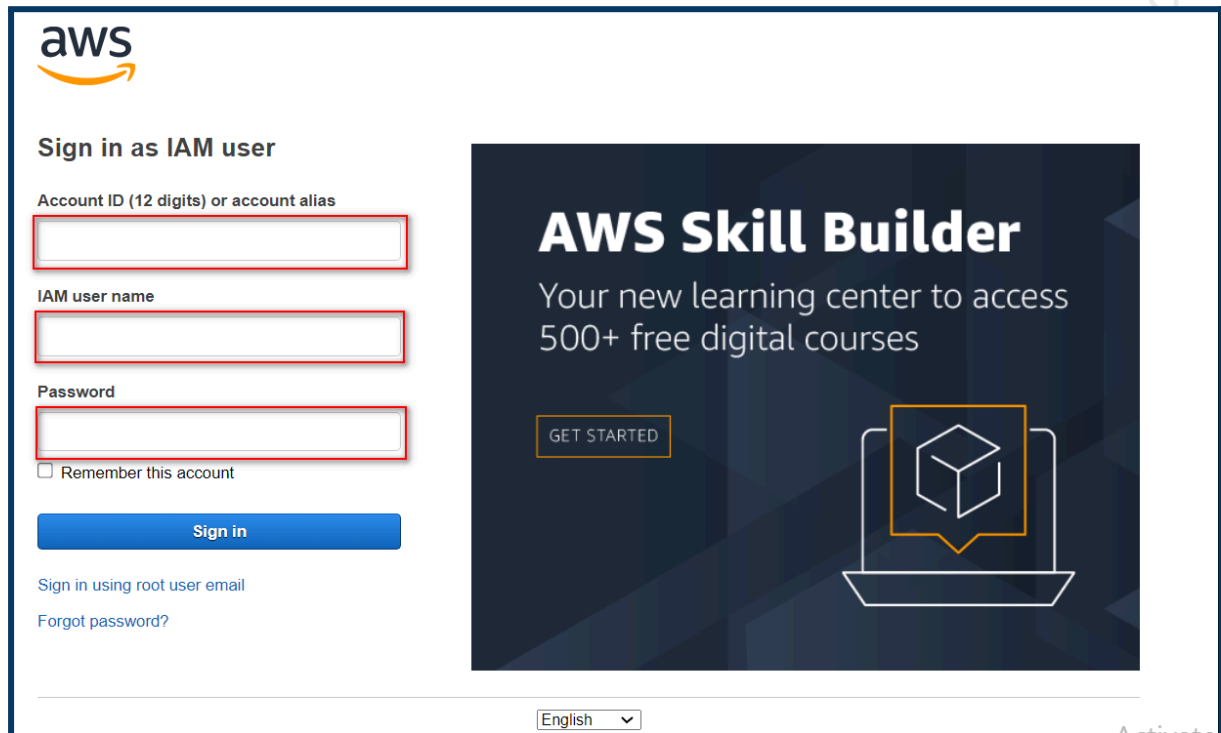


Figure 1.1: Search result

 **Note:** Log in with your credentials. (**Always user IAM user to login** followed by Account ID and password).

 **Learn More (IAM user):** IAM (Identity and Access Management) users in AWS are entities that you create in AWS that are associated with specific permissions to access and manage AWS services and resources. These users are separate from your AWS account and allow you to grant or restrict access to AWS resources for individuals, applications, or services. You can also learn more from the IAM [User Guide](#).

b. Navigate to EC2 Dashboard:

Click on the "Services" dropdown in the top left corner.

Select "EC2" under the "Compute" section. (Figure 1.2: EC2 Search result)

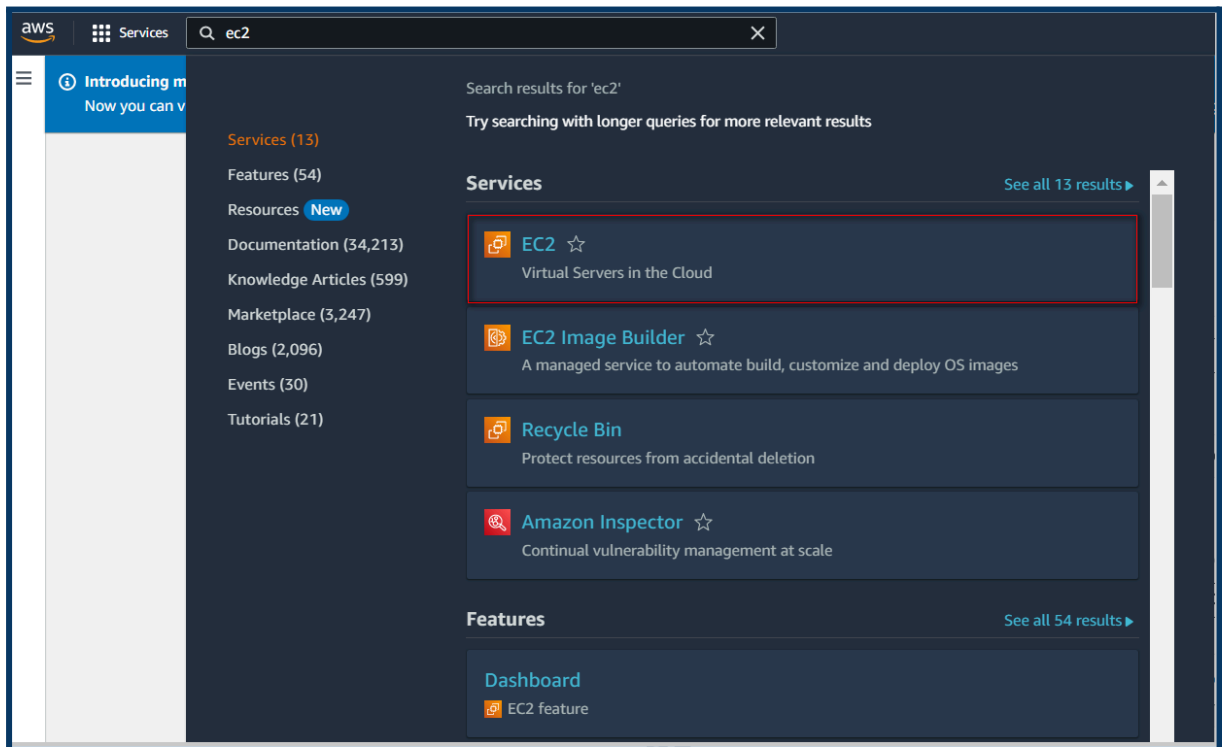


Figure 1.2: EC2 Search result

c. Launch Instance:

Click on the "Instances" link in the EC2 Dashboard.

Press the "Launch Instance" button. (Figure 1.3: Launch Instance Tab)

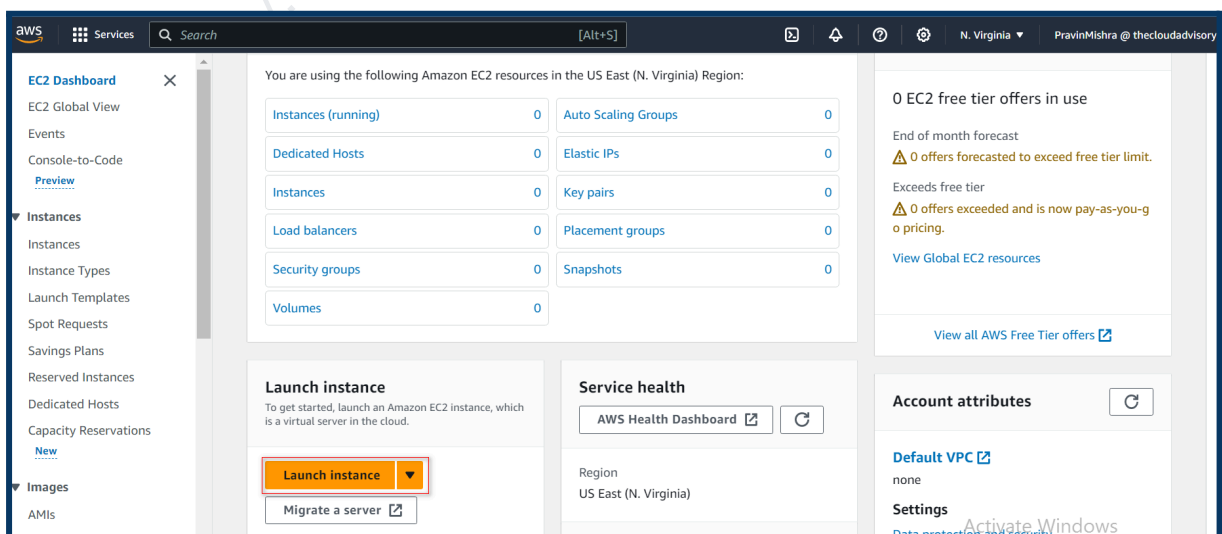


Figure 1.3: Launch Instance Tab

Name the instance you are going to create!

d. Choose an Amazon Machine Image (AMI):

Select "Amazon Linux 2 AMI" from the list of available AMIs.

Click on "Select" to proceed. *Figure 1.4: Choose an Amazon Machine Image (AMI)*

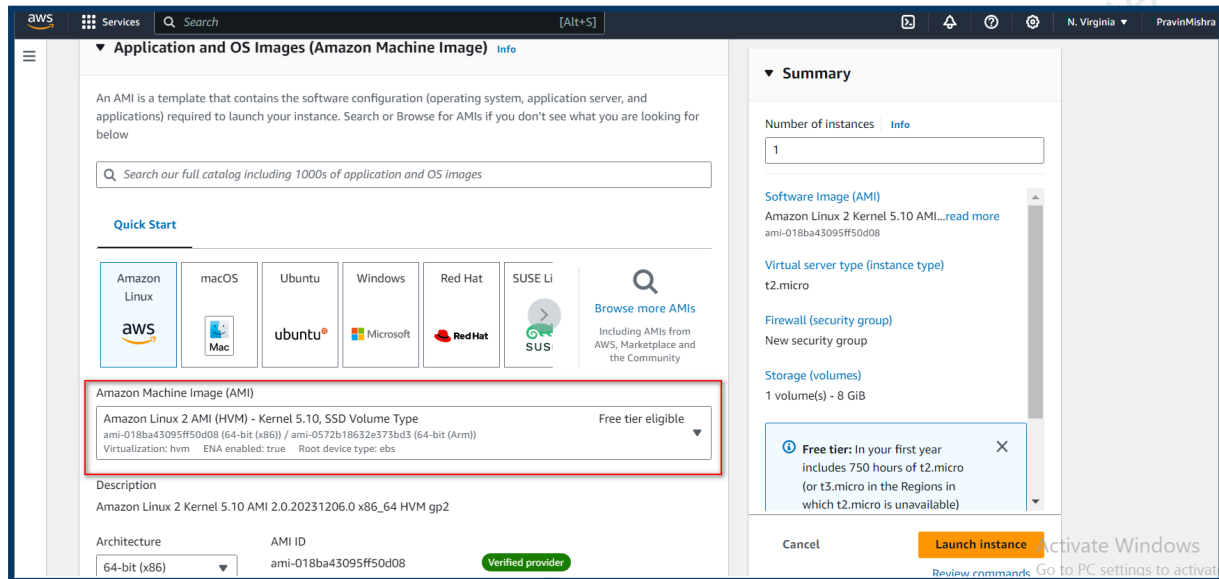


Figure 1.4: Choose an Amazon Machine Image (AMI)

✓ **Expected Output:** Successful selection of the instance type.

💡 **Consider:** Always ensure that the selected AMI aligns with your specific application or system requirements.

i Learn More (AMI): AMI stands for **Amazon Machine Image**. It's a pre-configured template that contains the necessary information to launch an instance, which is a virtual server in the Amazon Elastic Compute Cloud (EC2). AMIs include the **operating system, application server**, and any additional software needed to deploy an application. Read more about [AWS Documentation on Amazon Machine Images \(AMIs\)](#)

e. Choose t3.micro as the instance type for this exercise.
(*Figure 1.5: Choose an Instance Type*)

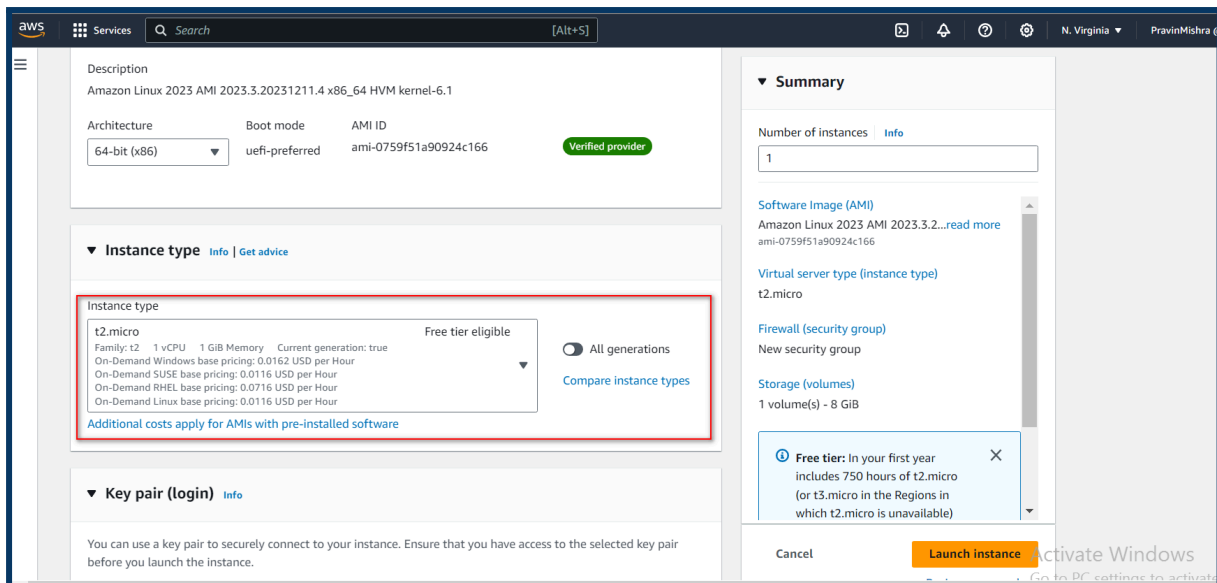


Figure 1.5: Choose an Instance Type

✓ **Expected Output:** Successful selection of the instance type.

💡 **Consider:** Always ensure that the chosen instance type (t3.micro) meets the resource requirements and capacity needed for your specific use case or workload.

i Learn More (Instance Type): **Instance Type**, refers to the **configuration and specifications of the virtual machine you'll deploy in AWS**. The t3.micro instance type is part of the T3 family, designed to offer a balance of compute, memory, and network resources suitable for various workloads. Read more about [AWS EC2 Instance Types](#)

STEP 2: Key Pair Creation:

- Create a new key pair during the instance setup process. This key pair will be used to securely SSH into the EC2 instance. (Figure 1.6: Key Pair Creation)

- Select "Create a new key pair" in the Key pair(login).
- Enter a name for the key pair.
- Download the key pair file (.pem) and keep it secure.

Create key pair Summary X

Key pair name Access to the selected key pair Number of instances Info

Key pairs allow you to connect to your instance securely.

EpicReads_Key

The name can include upto 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ **RSA**
RSA encrypted private and public key pair

☐ **ED25519**
ED25519 encrypted private and public key pair

Private key file format

☒ **.pem**
For use with OpenSSH

☐ **.ppk**
For use with PuTTY

Warning: When prompted, store the private key in a secure and accessible location on your computer. **You will need it later to connect to your instance.** [Learn more](#)

Cancel **Create key pair**

Figure 1.6: Key Pair Creation

✅ **Expected Output:** A newly generated key pair file (.pem) downloaded and stored securely.

📖 **Note:** Key pairs in AWS are used for secure access to instances running on Amazon EC2.

📘 **Learn More (Key pair Creation):** **Key pairs** in AWS are used for **secure access to instances** running on Amazon EC2. They consist of a public key that AWS stores, and a private key file that you can download to access your instances. Read more about [EC2 Key Pairs Documentation](#).

STEP 3: VPC and Network Configuration:

- Use an existing VPC, ensuring it's connected to a public subnet.
- Enable "Auto-assign public IP" to ensure the instance is accessible over the internet. (Figure 1.7: VPC and Network Configuration)

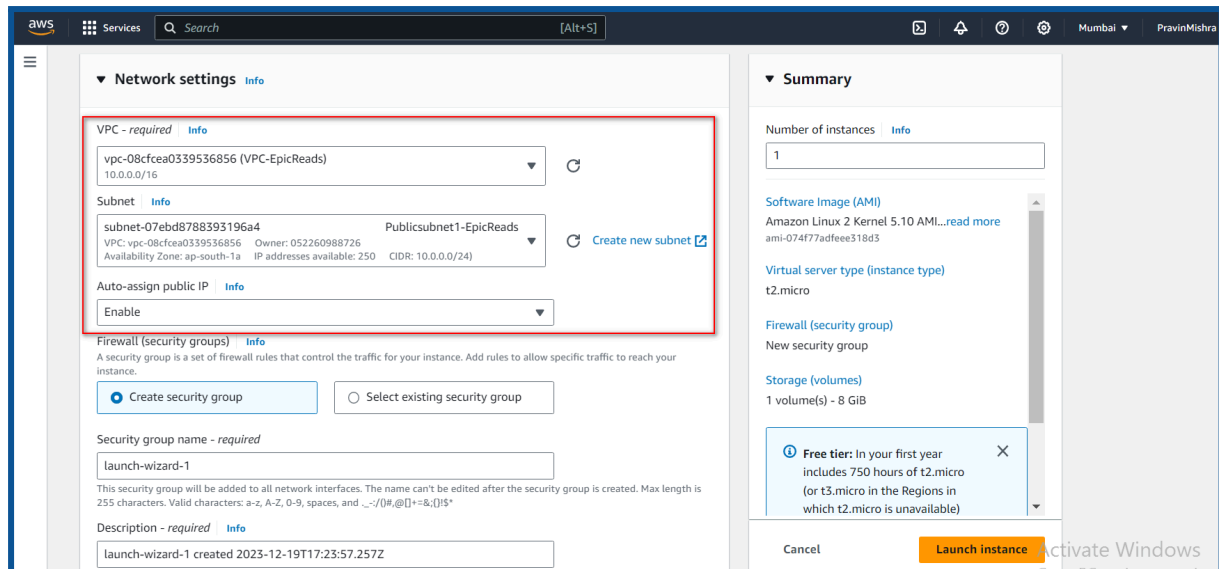


Figure 1.7: VPC and Network Configuration

✅ **Expected Output:** Confirmation that the VPC is connected to a public subnet and 'Auto-assign public IP' is enabled for internet accessibility.

i Learn More (VPC and Network Configuration): VPC (Virtual Private Cloud) in AWS allows you to create a **virtual network environment, including subnets, route tables, and security settings**, to launch AWS resources like EC2 instances..Read more about [AWS VPC User Guide](#).

STEP 4: Security Group Setup:

- Create a new security group for instance and specify the group name and description. (Figure 1.8: Security Group Setup)

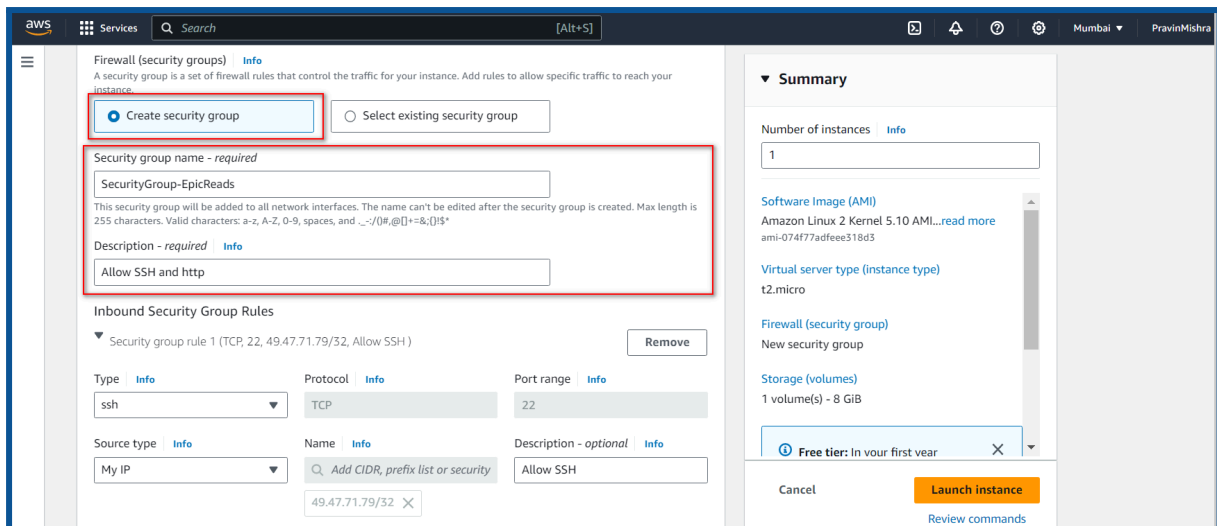


Figure 1.8: Security Group Setup

- b. Configure inbound rules to allow traffic on ports 22 (SSH) and 80 (HTTP) from the internet. (Figure 1.9: Security Group Rules).

Port 22:

Port 22 is similar to having a secret key that allows us to securely access and control our computer or server from anywhere, like opening a locked door with a special code. It helps us manage our computer or server even when we are far away.

Port 80:

Port 80 acts as the main entrance to the internet, just like the front door of a house where websites come and go. When we want people to see a website or application we have made, the information travels through this entrance.

In Amazon Web Services (AWS), we use port 22 as a special key to remotely access our computer or server (like using a secret code), and port 80 to showcase websites or applications on the internet (like the main entry point). We create a website and use Apache for a new project, it will need port 80 to display the application on the internet.

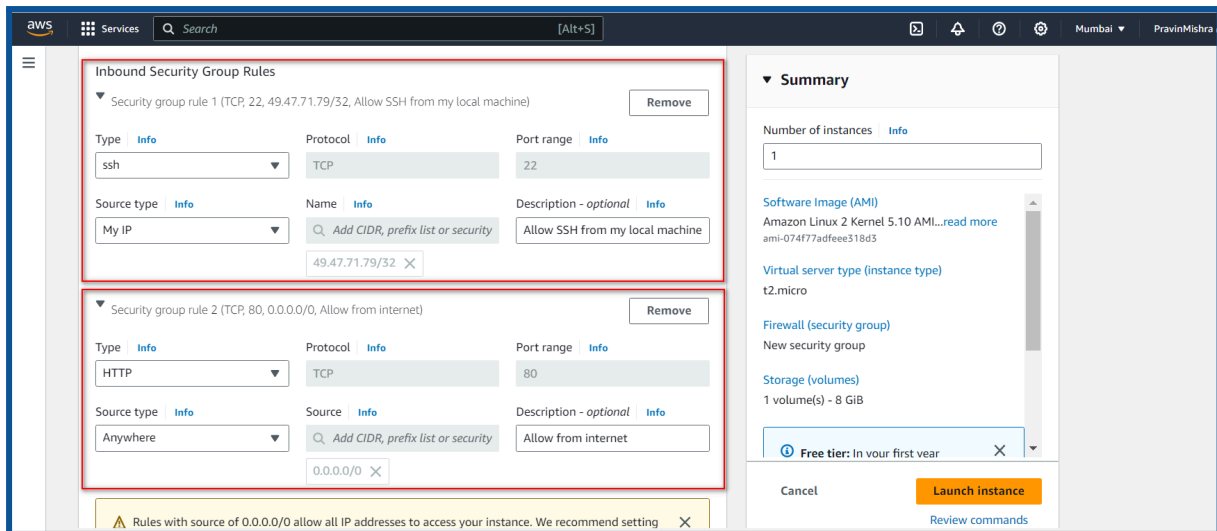


Figure 1.9: Security Group Rules

✅ **Expected Output:** Verification of newly configured security rules confirming SSH access limited to 'My IP' on port 22 and allow HTTP access from 'Anywhere' on port 80.

⚠️ **Caution:** It's crucial to accurately configure inbound rules to prevent potential security vulnerabilities or connectivity issues.

📘 **Learn More (Security Group Setup):** Security Groups in AWS act as virtual firewalls that control inbound and outbound traffic for AWS instances. Read more about [EC2 Security Groups Documentation](#).

STEP 5: Storage Configuration:

- Modify the instance's storage settings, setting the size to 10 GB. (Figure 1.10: Storage Configuration)

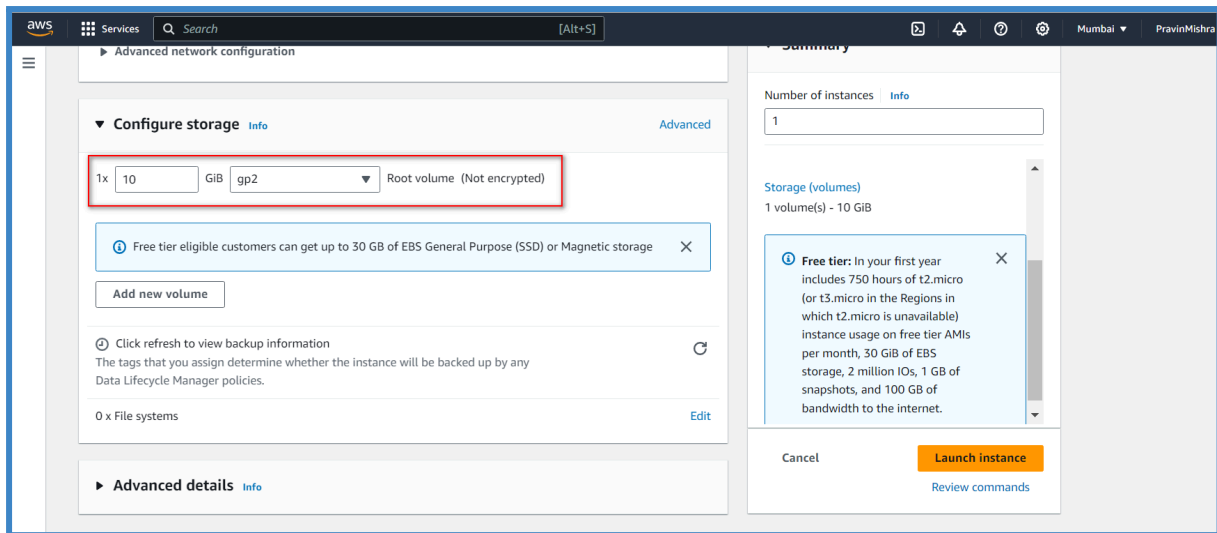


Figure 1.10: Storage Configuration

🚀 Fantastic work! You're moments away from launching your instance. Simply click on the 'Launch Instance' button to proceed with the final step." (Figure 1.10: Launch Instance Final)

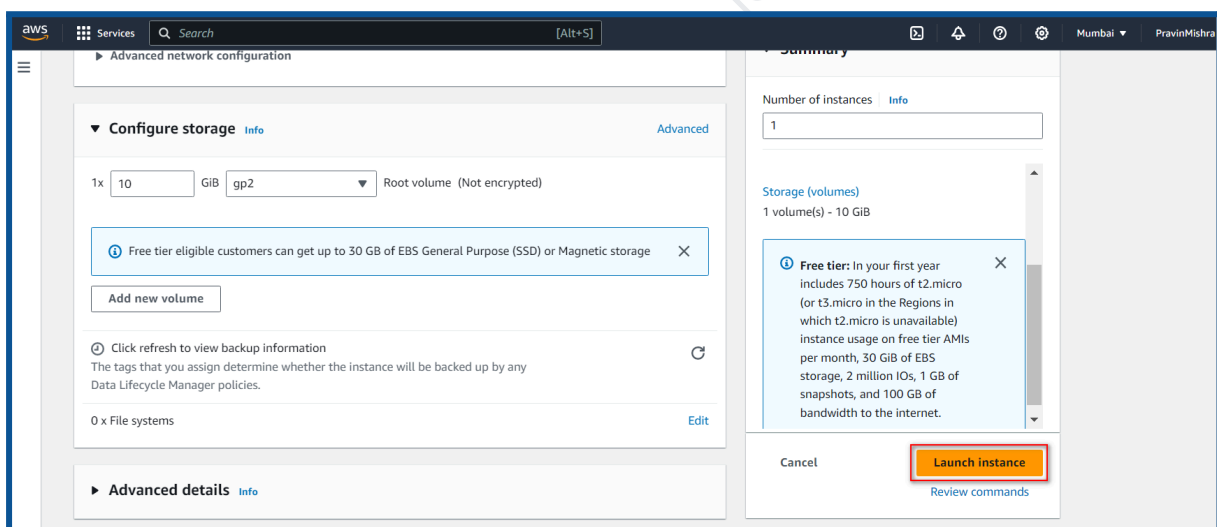


Figure 1.10: Launch Instance Final

STEP 6: SSH Connection and Testing:

- Select your EC2 instance: Click on the checkbox next to your EC2 instance to select it.
- Click on the "Connect" button: This should be located at the top of the EC2 Dashboard when you have your instance selected.
- Go to the SSH client section: Once you're in the "Connect" window, you should see an option for an SSH client. There, you'll find an example of the SSH connection string.

- d. Copy the connection string along with the username: In the SSH client section, you'll typically find a command that looks similar to this:
- ```
ssh -i /path/to/your-key.pem ec2-user@your-instance-public-ipv4
```
- (Figure 1.11: SSH Client Console)

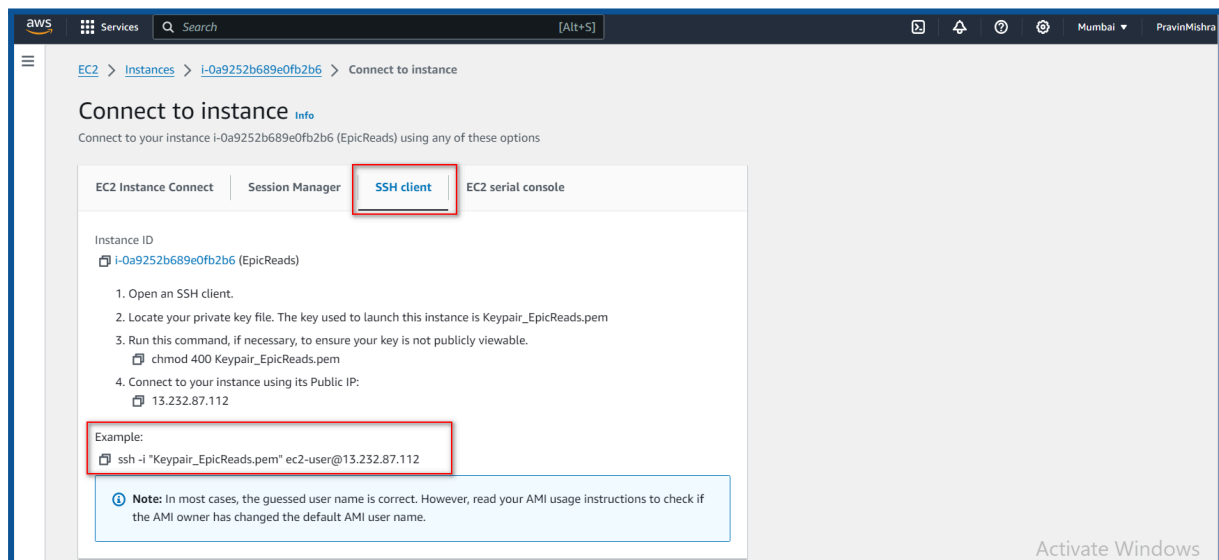


Figure 1.11: SSH Client Console

## STEP 7: Connecting via Command Line (Terminal):

- Open a terminal window on your local machine (Linux/macOS) or use an SSH client like PuTTY on Windows or Git Bash.
- Navigate to the directory where your private key (.pem) file is located: Use the `cd` command to navigate to the directory where your key is stored.  
For example: `cd /path/to/your-key-pair-directory`
- Change permissions for the private key: Ensure that the permissions for the private key are secure.

 **Command:** `chmod 400 your-key-pair.pem`

The command "`chmod 400 your-key-pair.pem`" helps protect a file called "`your-key-pair.pem`" by allowing only the owner to read it. This keeps the information inside safe and secure from others who shouldn't have access to it.

- d. Connect to the EC2 instance using SSH: Use the SSH command in the terminal to connect to your EC2 instance (*Figure 1.12: Connecting via Command Line*). The format is:

 **Command:** `ssh -i your-key-pair.pem ec2-user@your-instance-public-ipv4`

Example: `ssh -i "Keypair_EpicReads.pem" ec2-user@13.232.87.112`

- e. Once connected, perform a basic network test by pinging an external site like google.com.

 **Note:** Always ensure the correct path to the key pair file and instance public IP when connecting via SSH. (common errors)

```
ec2-user@ip-10-0-0-130:~
ut@lenovo MINGW64 ~
$ pwd
/c/Users/ut

ut@lenovo MINGW64 ~
$ cd downloads

ut@lenovo MINGW64 ~/downloads
$ chmod 400 Keypair_EpicReads.pem

ut@lenovo MINGW64 ~/downloads
$ ssh -i "Keypair_EpicReads.pem" ec2-user@13.232.87.112
Last login: Wed Dec 20 00:56:35 2023 from 49.47.69.249

#_#
Amazon Linux 2

AL2 End of Life is 2025-06-30.

V~'.'->
A newer version of Amazon Linux is available!

Amazon Linux 2023, GA and supported until 2028-03-15.
https://aws.amazon.com/linux/amazon-linux-2023/

3 package(s) needed for security, out of 3 available
Run "sudo yum update" to apply all updates.
[ec2-user@ip-10-0-0-130 ~]$ ping google.com
PING google.com (142.250.183.206) 56(84) bytes of data:
64 bytes from bom07s33-in-f14.1e100.net (142.250.183.206): icmp_seq=1 ttl=51 time=1.48 ms
64 bytes from bom07s33-in-f14.1e100.net (142.250.183.206): icmp_seq=2 ttl=51 time=1.53 ms
64 bytes from bom07s33-in-f14.1e100.net (142.250.183.206): icmp_seq=3 ttl=51 time=1.45 ms
64 bytes from bom07s33-in-f14.1e100.net (142.250.183.206): icmp_seq=4 ttl=51 time=1.48 ms
64 bytes from bom07s33-in-f14.1e100.net (142.250.183.206): icmp_seq=5 ttl=51 time=1.50 ms
64 bytes from bom07s33-in-f14.1e100.net (142.250.183.206): icmp_seq=6 ttl=51 time=1.48 ms
64 bytes from bom07s33-in-f14.1e100.net (142.250.183.206): icmp_seq=7 ttl=51 time=1.45 ms
^C
--- google.com ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6009ms
```

*Figure 1.12: Connecting via Command Line*

✅ **Expected Output:** Successful SSH connection established to the EC2 instance using the provided key pair and IP, followed by a successful network test by pinging an external site like google.com.

***Congratulations on launching your first EC2 instance!***

## Task 3 : Launching, Configuring, and Installing WordPress on a Linux EC2 Instance at EpicReads

### Task Overview:

Once you've successfully launched and connected to your Linux EC2 instance, the subsequent step involves installing and configuring WordPress. This will replicate a real-life scenario, allowing EpicReads to oversee digital content, blogs, or an online catalog of books.

This guide covers the following:

1. Installing and configuring a WordPress application on the Amazon Linux 2 EC2 instance.
2. Verifying that the WordPress site is accessible and fully functional from the internet.

## COMMON ERRORS

### 1. YUM Update Failure:

- **Error:** `sudo yum update` fails to update packages due to network issues, conflicting repositories, or missing dependencies.
- **Solution:** Check network connectivity, verify repositories, resolve dependency issues, and rerun the command. Use `sudo yum clean all` to clear the package cache and try updating again.

### 2. Apache Installation Failure:

- **Error:** `sudo yum install httpd -y` fails due to repository configuration problems, incorrect package names, or interrupted installation.
- **Solution:** Ensure repositories are configured correctly. Verify the package name (`httpd` in this case). Recheck the command and ensure it completes without interruptions.

### 3. Apache Service Not Starting:

- **Error:** `sudo systemctl start httpd` or `sudo systemctl status httpd` commands show errors or indicate that the service is inactive.

- **Solution:** Check logs (`sudo journalctl -xe` or Apache logs) for errors. Verify the configuration files and permissions. Rectify any issues and try restarting the service.
4. **Browser Doesn't Display Apache Test Page:**
    - **Error:** Entering the EC2 instance's public IP address in the browser doesn't load the Apache test page or shows an error.
    - **Solution:** Verify the EC2 instance's security group settings; ensure that HTTP (port 80) traffic is allowed. Double-check the public IP address used in the browser. Confirm that the Apache service is running and properly configured.
  5. **Misconfigured EC2 Instance:**
    - **Error:** Misconfigurations in the EC2 instance settings or network settings prevent Apache from displaying the test page.
    - **Solution:** Review EC2 instance settings, networking (VPC, subnet, route tables), and ensure the instance is properly associated with the desired security groups.
  6. **Misconfigured Inbound Rules:**
    - **Error:** Misconfiguring the inbound rules, such as selecting the wrong type (e.g., choosing "SSH" instead of "MYSQL/Aurora") or setting the wrong source.
    - **Solution:** Double-check the rules and ensure the correct type (MYSQL/Aurora, HTTP) is selected. Confirm the source addresses (e.g., "My IP" or specific IP ranges) are set accurately.
  7. **Missing Database Configuration:**
    - **Error:** Incorrect MySQL database settings in `wp-config.php`.
    - **Solution:** Double-check database details (name, username, password, host) and ensure they match the actual database setup.

## STEP 1: Update and Upgrade Packages

Once connected to your EC2 instance, update the package lists and upgrade installed packages by using `sudo yum update`: (*Figure 2.1: Update and Upgrade Packages*).

 **Command:**

```
sudo yum update
```



```
ec2-user@ip-10-0-0-130:~
ut@lenovo MINGW64 ~/downloads
$ ssh -i "Keypair_EpicReads.pem" ec2-user@65.0.93.46
Last login: Thu Dec 21 04:15:00 2023 from 49.47.70.122

Amazon Linux 2

AL2 End of Life is 2025-06-30.

A newer version of Amazon Linux is available!

Amazon Linux 2023, GA and supported until 2028-03-15.
https://aws.amazon.com/linux/amazon-linux-2023/
[ec2-user@ip-10-0-0-130 ~]$ sudo yum install httpd -y
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
amzn2-core
Resolving Dependencies
--> Running transaction check
--> Package httpd.x86_64 0:2.4.58-1.amzn2 will be installed
--> Processing Dependency: httpd filesystem = 2.4.58-1.amzn2 for package: httpd-2.4.58-1.amzn2.x86_64
--> Processing Dependency: httpd-tools = 2.4.58-1.amzn2 for package: httpd-2.4.58-1.amzn2.x86_64
--> Processing Dependency: /etc/mime.types for package: httpd-2.4.58-1.amzn2.x86_64
--> Processing Dependency: httpd filesystem for package: httpd-2.4.58-1.amzn2.x86_64
--> Processing Dependency: mod_http2 for package: httpd-2.4.58-1.amzn2.x86_64
--> Processing Dependency: system-logos-httpd for package: httpd-2.4.58-1.amzn2.x86_64
--> Processing Dependency: libapr-1.so.0(64bit) for package: httpd-2.4.58-1.amzn2.x86_64
--> Processing Dependency: libaprutil-1.so.0(64bit) for package: httpd-2.4.58-1.amzn2.x86_64
--> Running transaction check
--> Package apr.x86_64 0:1.7.2-1.amzn2 will be installed
--> Package apr-util.x86_64 0:1.6.3-1.amzn2.0.1 will be installed
--> Processing Dependency: apr-util-bdb(x86-64) = 1.6.3-1.amzn2.0.1 for package: apr-util-1.6.3-1.amzn2.0.1.x86_64
--> Package generic-logos-httpd.noarch 0:18.0.0-4.amzn2 will be installed
--> Package httpd filesystem.noarch 0:2.4.58-1.amzn2 will be installed
--> Package httpd-tools.x86_64 0:2.4.58-1.amzn2 will be installed
--> Package mailcap.noarch 0:2.1.41-2.amzn2 will be installed
--> Package mod_http2.x86_64 0:1.15.19-1.amzn2.0.1 will be installed
--> Running transaction check
--> Package apr-util-bdb.x86_64 0:1.6.3-1.amzn2.0.1 will be installed
--> Finished Dependency Resolution
Dependencies Resolved
```

Figure 2.2: Install Apache Web Server

Start the Apache service and enable it to start on boot (Figure 2.3: Apache service status)

#### Command:

```
sudo systemctl start httpd
sudo systemctl status httpd
```

```
ec2-user@ip-10-0-0-130:~
[ec2-user@ip-10-0-0-130 ~]$
[ec2-user@ip-10-0-0-130 ~]$ sudo systemctl start httpd
[ec2-user@ip-10-0-0-130 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
 Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor preset: disabled)
 Active: active (running) since Thu 2023-12-21 06:35:13 UTC; 20h ago
 Docs: man:httpd.service(8)
 Main PID: 3945 (httpd)
 Status: "Total requests: 88; Idle/Busy workers 100/0; Requests/sec: 0.00121; Bytes served/sec: 2 B/sec"
 CGroup: /system.slice/httpd.service
 └─3945 /usr/sbin/httpd -DFOREGROUND
 └─3946 /usr/sbin/httpd -DFOREGROUND
 └─3947 /usr/sbin/httpd -DFOREGROUND
 └─3948 /usr/sbin/httpd -DFOREGROUND
 └─3949 /usr/sbin/httpd -DFOREGROUND
 └─3950 /usr/sbin/httpd -DFOREGROUND
 └─4079 /usr/sbin/httpd -DFOREGROUND
 └─4511 /usr/sbin/httpd -DFOREGROUND
Dec 21 06:35:13 ip-10-0-0-130.ap-south-1.compute.internal systemd[1]: Starting The Apache HTTP Server...
Dec 21 06:35:13 ip-10-0-0-130.ap-south-1.compute.internal systemd[1]: Started The Apache HTTP Server.
[ec2-user@ip-10-0-0-130 ~]$
```

Figure 2.3: Apache service status

✓ **Expected Output:** `sudo systemctl status httpd` should indicate that the Apache service is active and running without any errors.

To verify the deployment of Apache HTTPD through the browser:

- Obtain the public IP address of your EC2 instance (*Figure 2.4: Public IP address*).

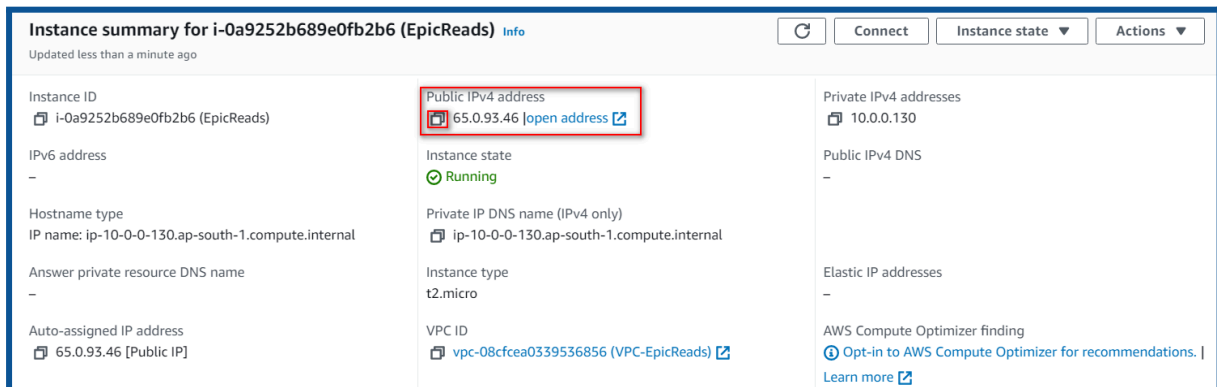


Figure 2.4: Public IP address

- Open your web browser.
- Enter the EC2 instance's public IP address into the browser's address bar.
- Press "Enter" to load the page (*Figure 2.5: Apache Test Page*).

Example: `http://[EC2_Public_IP]`

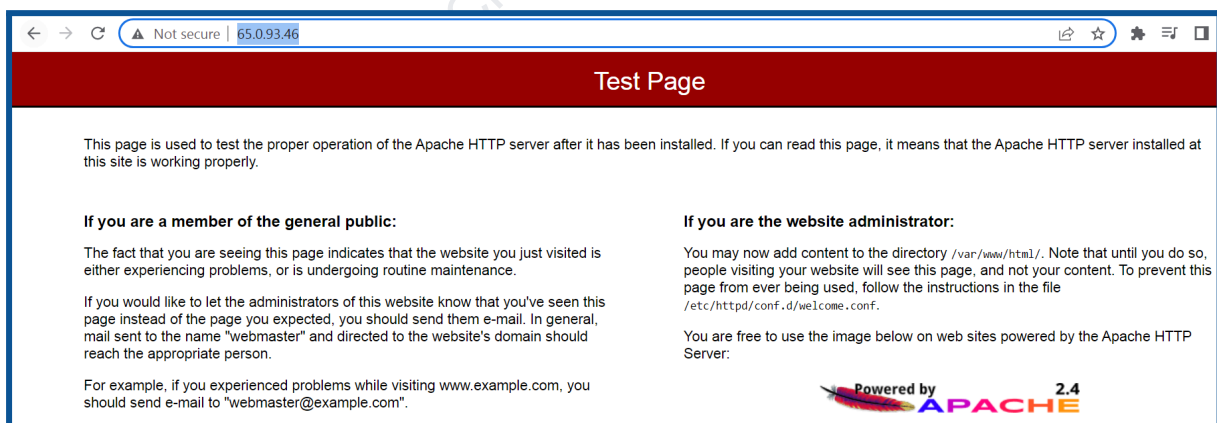


Figure 2.5: Apache Test Page

✓ **Expected Output:** The the expected output in the browser should display the default Apache test page, indicating that Apache HTTPD has been successfully deployed and is functioning correctly.

📌 **Note:** The appearance of the Apache test page confirms the **successful deployment of Apache HTTPD on your EC2 instance**. If

the test page does not load, ensure that the installation and configuration steps have been executed correctly. Troubleshoot any issues by revisiting the installation commands and verifying the server status.

**i Learn More:** To dive deeper into the Apache HTTP Server, AWS provides additional information about [setting up Apache on Amazon Linux](#).

## STEP 3: Install PHP and MySQL

Install PHP along with necessary PHP extensions required by WordPress(*Figure 2.6: install PHP*).

### Command:

```
sudo amazon-linux-extras install -y lamp-mariadb10.2-php7.2 php7.2
```

```
[ec2-user@ip-10-0-1-38 wordpress]$ sudo amazon-linux-extras install -y lamp-mariadb10.2-php7.2 php7.2
Topic lamp-mariadb10.2-php7.2 has end-of-support date of 2020-11-30
Topic php7.2 has end-of-support date of 2020-11-30
Installing php-pdo, php-mysqlnd, php-fpm, php-cli, php-json, mariadb
Failed to set locale, defaulting to C
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
Cleaning repos: amzn2-core amzn2extra-docker amzn2extra-kernel-5.10
 : amzn2extra-lamp-mariadb10.2-php7.2 amzn2extra-php7.2
17 metadata files removed
6 sqlite files removed
0 metadata files removed
Failed to set locale, defaulting to C
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
amzn2-core | 3.6 kB | 00:00
amzn2extra-docker | 2.9 kB | 00:00
amzn2extra-kernel-5.10 | 3.0 kB | 00:00
amzn2extra-lamp-mariadb10.2-php7.2 | 3.0 kB | 00:00
amzn2extra-php7.2 | 3.0 kB | 00:00
(1/11): amzn2-core/2/x86_64/group_gz | 2.7 kB | 00:00
(2/11): amzn2-core/2/x86_64/updateinfo | 787 kB | 00:00
(3/11): amzn2extra-docker/2/x86_64/updateinfo | 14 kB | 00:00
(4/11): amzn2extra-docker/2/x86_64/primary_db | 105 kB | 00:00
(5/11): amzn2extra-lamp-mariadb10.2-php7.2/2/x86_64/update | 76 B | 00:00
(6/11): amzn2extra-php7.2/2/x86_64/updateinfo | 76 B | 00:00
(7/11): amzn2extra-kernel-5.10/2/x86_64/updateinfo | 44 kB | 00:00
(8/11): amzn2extra-lamp-mariadb10.2-php7.2/2/x86_64/primar | 506 kB | 00:00
(9/11): amzn2extra-kernel-5.10/2/x86_64/primary_db | 22 MB | 00:00
```

*Figure 2.6: install PHP*

This command installs PHP along with various extensions that are essential for WordPress to function properly. Here's a breakdown of the extensions installed:

- php-mysqlnd: Enables PHP to communicate with MySQL databases.
- php-gd: Provides image manipulation functionalities for WordPress.

- php-xml: Allows PHP to handle XML data.
- php-mbstring: Necessary for multibyte string support in PHP.

Once these steps are completed, PHP will be installed and ready to support your WordPress installation on your system.

✅ **Expected Output:** This command installs PHP along with essential extensions necessary for **WordPress functionality**. No specific output is expected if the installation is successful.

📖 **Note:** These PHP extensions provide critical functionalities to WordPress. They enable PHP to communicate with MySQL, handle XML data, support multibyte strings, and perform image manipulation tasks.

ℹ️ **Learn More:** For further details about PHP and its modules, you can refer to the AWS documentation on [PHP on Amazon Linux](#).

## Install MySQL:

Install MySQL server package for the WordPress database(*Figure 2.7: install MySQL*).

💻 **Command:**

```
sudo yum install -y mysql
```



```
cd /var/www/html/
```

b. Download WordPress:

- Fetch the latest version of WordPress.

 **Command:**

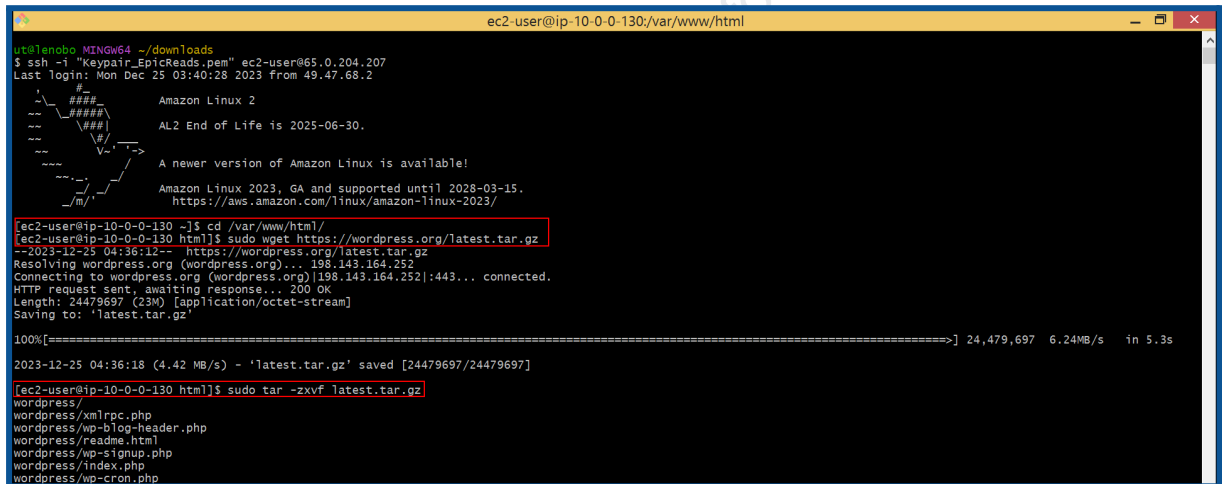
```
sudo wget https://wordpress.org/latest.tar.gz
```

c. Extract WordPress Archive:

- Unzip the downloaded WordPress file.

 **Command:**

```
sudo tar -xzf latest.tar.gz
```



```
ec2-user@ip-10-0-0-130:/var/www/html
ut@lenovo MINGW64 ~/downloads
$ ssh -i "keypair_EpicReads.pem" ec2-user@65.0.204.207
Last login: Mon Dec 25 03:40:28 2023 from 49.47.68.2

#####
-- Amazon Linux 2
-- #####
-- AL2 End of Life is 2025-06-30.
-- #####
-- A newer version of Amazon Linux is available!
-- Amazon Linux 2023, GA and supported until 2028-03-15.
-- https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-0-0-130 ~]$ cd /var/www/html/
[ec2-user@ip-10-0-0-130 html]$ sudo wget https://wordpress.org/latest.tar.gz
--2023-12-25 04:36:12-- https://wordpress.org/latest.tar.gz
Resolving wordpress.org (wordpress.org)... 198.143.164.252
Connecting to wordpress.org (wordpress.org)|198.143.164.252|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 24479697 (23M) [application/octet-stream]
Saving to: 'latest.tar.gz'

100%[=====] 24,479,697 6.24MB/s in 5.3s

2023-12-25 04:36:18 (4.42 MB/s) - 'latest.tar.gz' saved [24479697/24479697]

[ec2-user@ip-10-0-0-130 html]$ sudo tar -xzf latest.tar.gz
wordpress/
wordpress/xmlrpc.php
wordpress/wp-blog-header.php
wordpress/readme.html
wordpress/wp-signup.php
wordpress/index.php
wordpress/wp-cron.php
```

Figure 2.8: Download WordPress

 **Expected Output:** Successful downloading of the WordPress archive (*latest.tar.gz*) indicated by completion message.

 **Note:** Ensure you download WordPress from a reliable source to avoid issues with the installation package.

 **Learn More:** For detailed information about downloading WordPress, refer to AWS documentation on [Installing WordPress on Amazon Linux](#).

d. Create WordPress Configuration(*Figure 2.9: Permission for Wordpress file*).

- Navigate into the WordPress directory.

### **Command:**

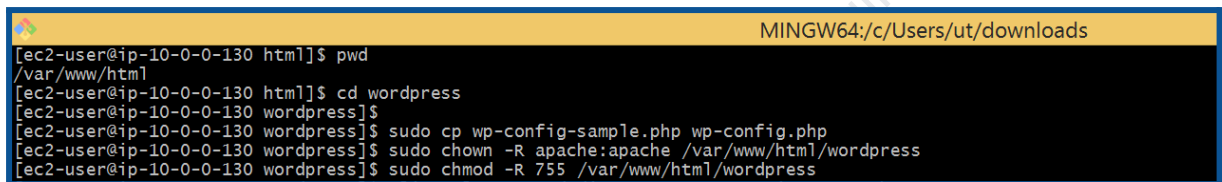
```
cd wordpress
```

Create the configuration file by copying the sample configuration.

```
sudo cp wp-config-sample.php wp-config.php
```

Set Permissions: Adjust the permissions for WordPress files.

```
sudo chown -R apache:apache /var/www/html/wordpress
sudo chmod -R 755 /var/www/html/wordpress
```



```
MINGW64:/c:/Users/ut/downloads
[ec2-user@ip-10-0-0-130 html]$ pwd
/var/www/html
[ec2-user@ip-10-0-0-130 html]$ cd wordpress
[ec2-user@ip-10-0-0-130 wordpress]$
[ec2-user@ip-10-0-0-130 wordpress]$ sudo cp wp-config-sample.php wp-config.php
[ec2-user@ip-10-0-0-130 wordpress]$ sudo chown -R apache:apache /var/www/html/wordpress
[ec2-user@ip-10-0-0-130 wordpress]$ sudo chmod -R 755 /var/www/html/wordpress
```

Figure 2.9: Permission for Wordpress file

 **Expected Output:** Successful creation of the WordPress configuration file (*wp-config.php*) and adjustment of permissions on WordPress files without any errors.

 **Note:** Proper file ownership and permissions are crucial for WordPress functionality. Ensure these settings are correctly applied.

## STEP 5: Create Wordpress Database

Before even creating the Database, creating a subnet group is important

Navigate to the left navigation pane of the RDS dashboard and click on the *Subnet groups* (Figure 2.10: Create Subnet groups)

Figure 2.10: Create Subnet groups

Navigate to the RDS console and click on "Create Database."(Figure 2.10:Create Database Tab)

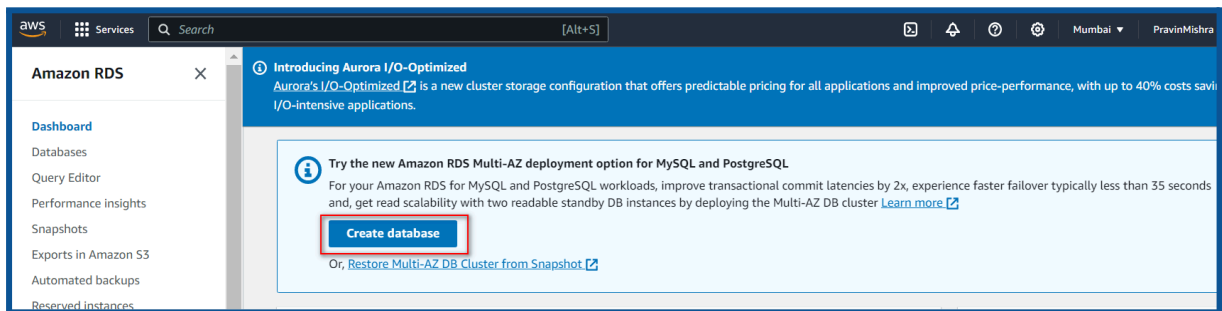


Figure 2.10:Create Database Tab

Under Engine options, choose "MySQL." (Figure 2.11:Create Database Option)

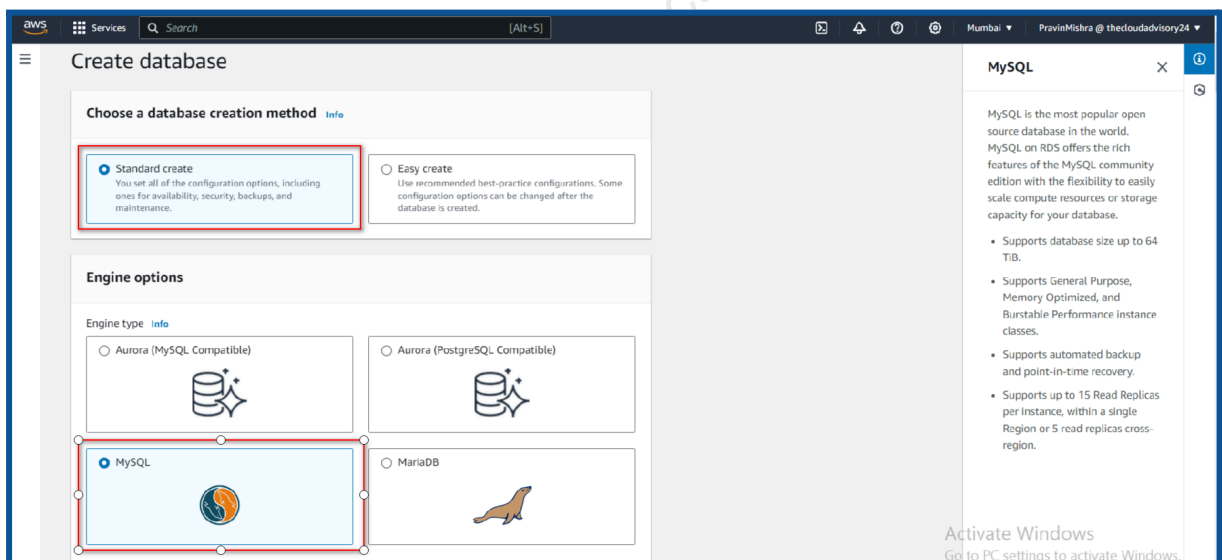


Figure 2.11:Create Database Option

Select the "Free tier" template in the Templates section(Figure 2.12: Free Tier Template)

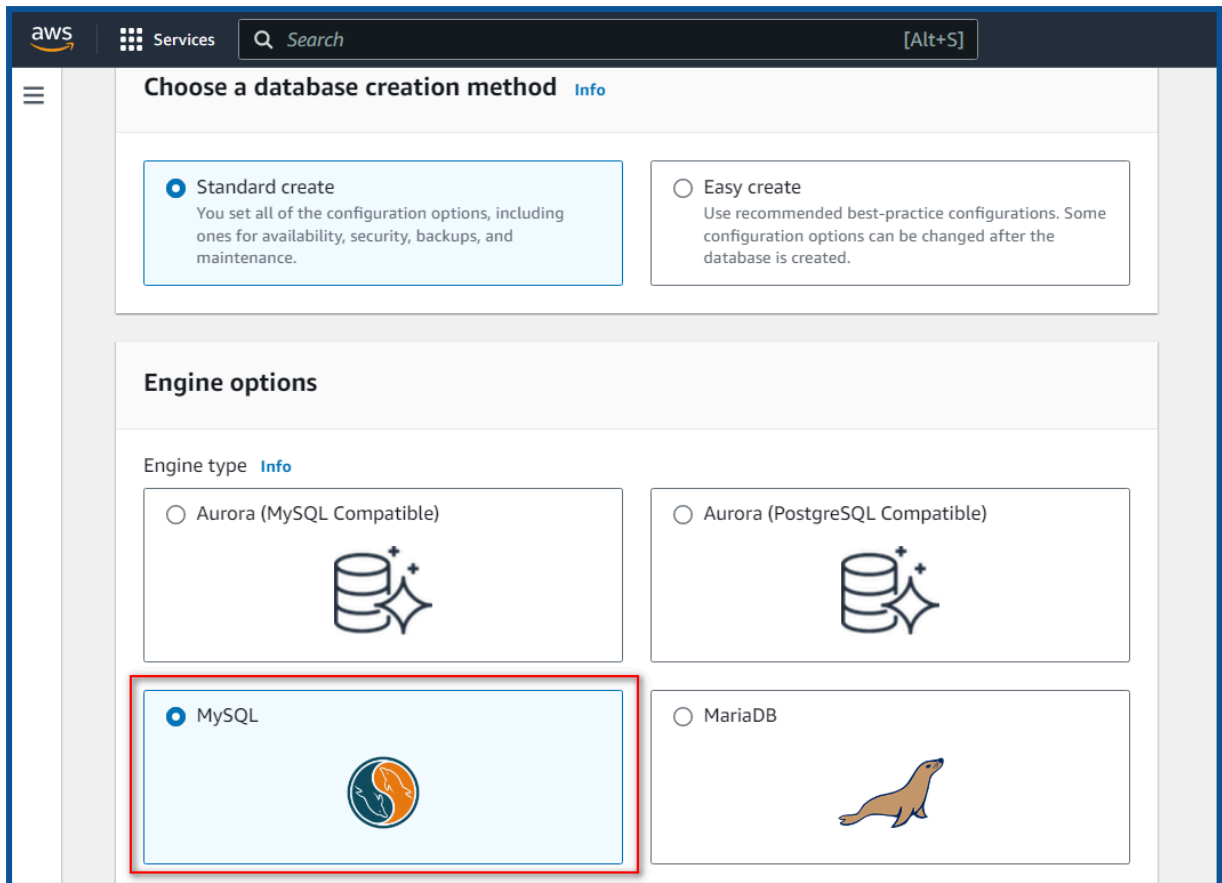


Figure 2.12: Free Tier Template

In the Settings section (Figure 2.13: Credentials Settings):

- Enter "wordpress" as the DB identifier.
- Set your desired Master username.
- Define a secure Master password (e.g. Happynewyear#24) for access and also enter the same password on the confirm password tab.

**Settings**

**DB instance identifier** [Info](#)  
Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.

wordpress

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 60 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

▼ **Credentials Settings**

**Master username** [Info](#)  
Type a login ID for the master user of your DB instance.

admin

1 to 16 alphanumeric characters. The first character must be a letter.

☐ **Manage master credentials in AWS Secrets Manager**  
Manage master user credentials in Secrets Manager. RDS can generate a password for you and manage it throughout its lifecycle.

**Info** If you manage the master user credentials in Secrets Manager, some RDS features aren't supported. [Learn more](#)

☐ **Auto generate a password**  
Amazon RDS can generate a password for you, or you can specify your own password.

**Master password** [Info](#)

.....

Constraints: At least 8 printable ASCII characters. Can't contain any of the following: / (slash), ' (single quote), " (double quote) and @ (at sign).

*Figure 2.13: Credentials Settings*

The instance configuration and storage should remain the same at their default selections(*Figure 2.14: Instance Configuration*).

- Use the default configuration for **Burstable Classes** with the **db.t3.micro** instance type.
- Specify the storage type as **General Purpose SSD (gp2)**.
- Allocate **20 GB** of storage capacity.

**Instance configuration**

The DB instance configuration options below are limited to those supported by the engine that you selected above.

DB instance class [Info](#)

▼ Hide filters

☐ Show instance classes that support Amazon RDS Optimized Writes [Info](#)  
Amazon RDS Optimized Writes improves write throughput by up to 2x at no additional cost.

☐ Include previous generation classes

☐ Standard classes (includes m classes)

☐ Memory optimized classes (includes r and x classes)

☒ Burstable classes (includes t classes)

db.t3.micro  
2 vCPUs 1 GiB RAM Network: 2,085 Mbps

**Storage**

Storage type [Info](#)

General Purpose SSD (gp2)  
Baseline performance determined by volume size

Allocated storage [Info](#)

20 GiB

The minimum value is 20 GiB and the maximum value is 6,144 GiB

*Figure 2.14: Instance Configuration*

In the Connectivity section(*Figure 2.15: Compute resource Configuration*):

- Choose the VPC named "VPC-EpicReads" created in the preceding step.
- For "Public access," select "No" to limit public access to the instance.

The screenshot shows the 'Connectivity' configuration page in the AWS Management Console. The page is divided into several sections:

- Compute resource:** Two radio buttons are present. The first, 'Don't connect to an EC2 compute resource', is selected. The second, 'Connect to an EC2 compute resource', is unselected.
- Virtual private cloud (VPC):** A dropdown menu is shown with 'VPC-EpicReads (vpc-08cfcea0339536856)' selected. Below the dropdown, it says '4 Subnets, 2 Availability Zones'. A red box highlights the dropdown menu.
- DB subnet group:** A dropdown menu is shown with 'Create new DB Subnet Group' selected.
- Public access:** Two radio buttons are present. The first, 'Yes', is unselected. The second, 'No', is selected. A red box highlights the 'No' radio button.

Informational messages are present in blue boxes:

- Under 'Virtual private cloud (VPC)': 'After a database is created, you can't change its VPC.'
- Under 'Public access': 'RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC security groups that specify which resources can connect to the database.'

Figure 2.15: Compute resource Configuration

Under "VPC security group," choose "Create new" and input "db-sg" as the name for the new VPC security group (Figure 2.16: VPC Security Group).

aws Services Search [Alt+S]

### VPC security group (firewall) [Info](#)

Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.

☐ Choose existing  
Choose existing VPC security groups

☒ Create new  
Create new VPC security group

New VPC security group name  
db-sg

Availability Zone [Info](#)  
No preference ▼

**RDS Proxy**  
RDS Proxy is a fully managed, highly available database proxy that improves application scalability, resiliency, and security.

☐ Create an RDS Proxy [Info](#)  
RDS automatically creates an IAM role and a Secrets Manager secret for the proxy. RDS Proxy has additional costs. For more information, see [Amazon RDS Proxy pricing](#).

**Certificate authority - optional** [Info](#)  
Using a server certificate provides an extra layer of security by validating that the connection is being made to an Amazon database. It does so by checking the server certificate that is automatically installed on all databases that you provision.

rds-ca-2019 (default)  
Expiry: Aug 22, 2024 ▼

If you don't select a certificate authority, RDS chooses one for you.

► Additional configuration

Figure 2.16: VPC Security Group

🔥 **WARNING:** Be cautious with the security group settings as it directly impacts network access to your database instance.

📖 **Note:** The security group configuration controls inbound and outbound traffic for your RDS instance.

Scroll down and click on "Additional connectivity configuration" to reveal further settings (*Figure 2.17: Additional Configuration*).

Enter "wordpress" as the initial database name.  
Disable backups and encryption, as they are not required for this project.

aws Services Search [Alt+S]

### Additional configuration

Database options, encryption turned off, backup turned off, backtrack turned off, maintenance, CloudWatch Logs, delete protection turned off.

#### Database options

Initial database name [Info](#)

wordpress

If you do not specify a database name, Amazon RDS does not create a database.

DB parameter group [Info](#)

default:mysql8.0

Option group [Info](#)

default:mysql-8-0

#### Backup

☐ Enable automated backups

Creates a point-in-time snapshot of your database

#### Encryption

☐ Enable encryption

Choose to encrypt the given instance. Master key IDs and aliases appear in the list after they have been created using the AWS Key Management Service console. [Info](#)

Figure 2.17: Additional Configuration

**Awesome!** We've reached the final step. Click 'Create database' to bring our database go live (Figure 2.18: Create Database Final Step)

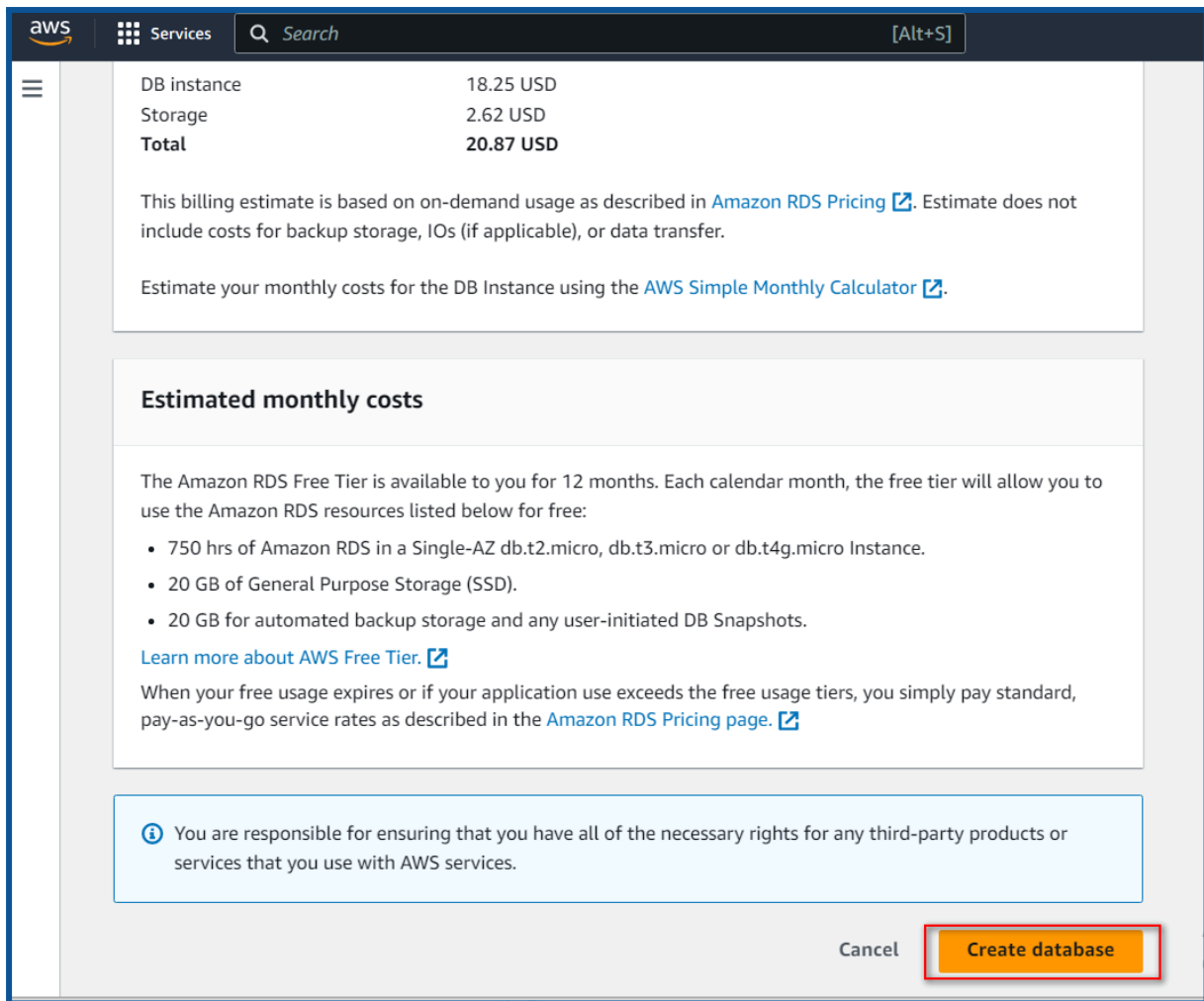


Figure 2.18: Create Database Final Step

Once created, you will see it on the RDS dashboard as shown in the below figure(Figure 2.19: RDS Dashboard).

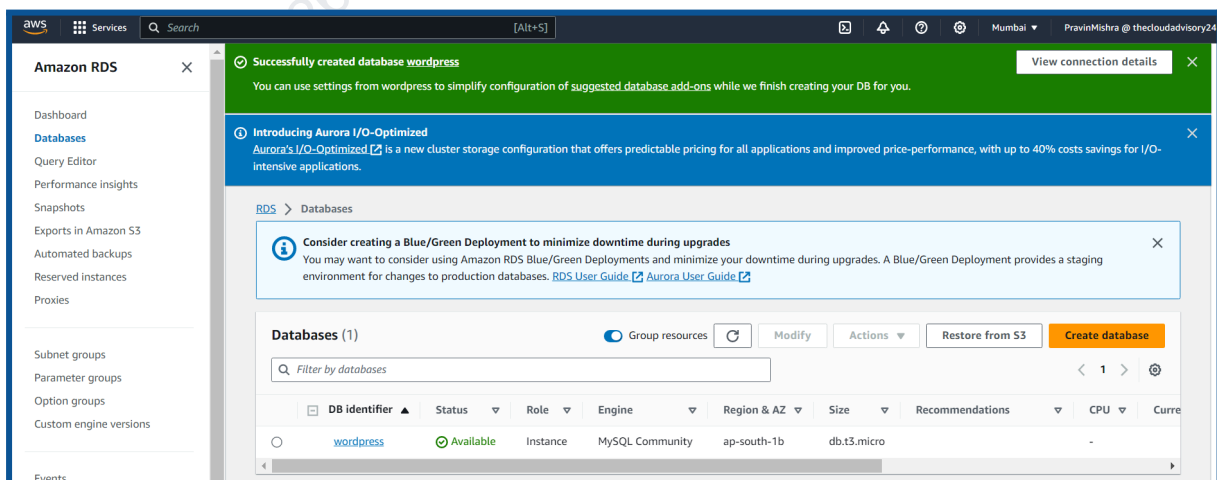


Figure 2.19: RDS Dashboard

✓ **Expected Output:** Successful creation of the database instance, visible on the RDS dashboard.

📖 **Note:** Review the RDS dashboard to verify the database instance's successful creation.

## STEP 6: Security Groups Settings

### 1. Visit Security Groups Page (Figure 2.20: Security Group EpicReads):

- Access the Security Groups page and choose "public-instance-sg."

#### a. Edit Inbound Rules:

- Click on the "Edit inbound rules" button.

#### b. Add MySQL/Aurora Rule:

- Click "Add rule."
- Choose "MYSQL/Aurora" for Type.
- Select "Custom" for Source and locate "db-sg," then save the rules.

#### c. Add HTTP Rule:

- Click "Add rule" again.
- Select "HTTP" for Type.
- Choose "My IP" for Source, and save the rules (Figure 2.17: Security Group EpicReads).

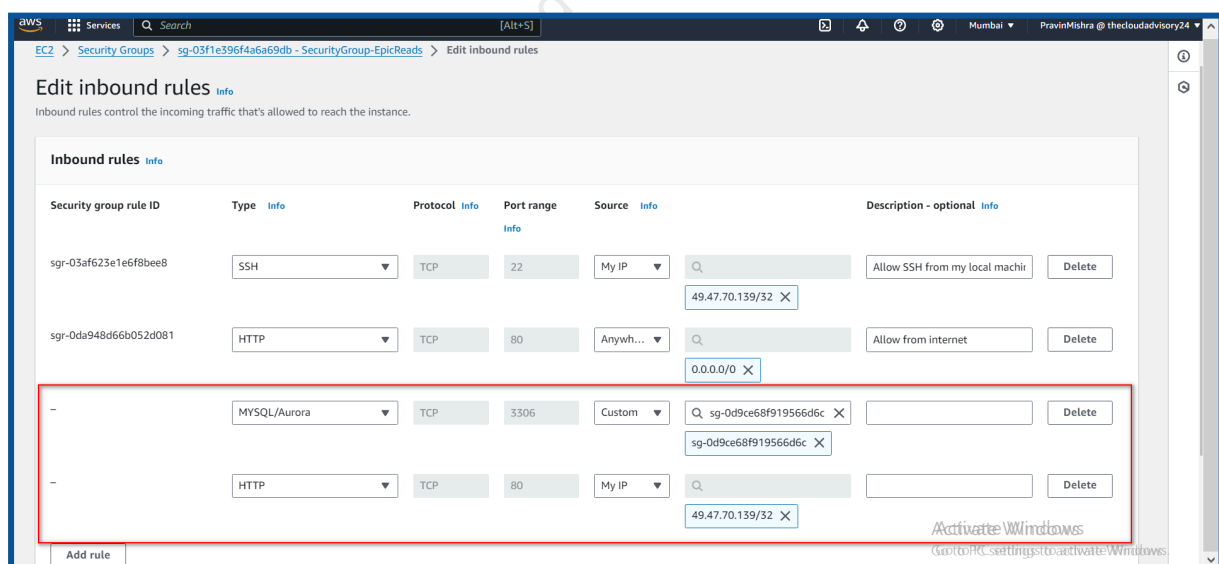


Figure 2.20: Security Group EpicReads

**Inbound Rules of Instance Security Group:** These rules often include allowing specific types of incoming traffic like HTTP (port 80) or SSH (port 22) from specific IP addresses or ranges. In the above steps, we added rules for MySQL/Aurora (port 3306) to allow communication between the

EC2 instance and the database server, as well as an HTTP rule to allow web traffic.

## 2. Access Security Groups of DB (Figure 2.21: Security Group DB):

- Navigate to the Security Groups page and choose "db-sg."

### a. Edit Inbound Rules:

- Click on the "Edit inbound rules" button.

### b. Delete Existing Rule:

- Remove any existing rule.

### c. Add MYSQL/Aurora Rule:

- Click "Add rule."
- Choose "MYSQL/Aurora" for Type.
- Select "Custom" for Source and locate "SecurityGroup-EpicReads," then save the rules (Figure 2.18: Security Group DB).

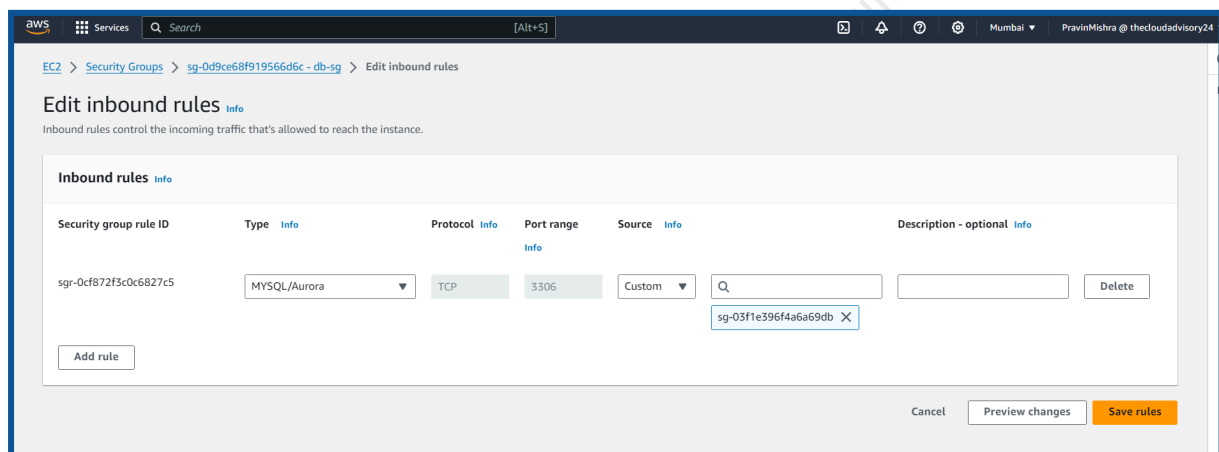


Figure 2.21: Security Group DB

**Inbound Rules of DB Security Group:** These rules define the source of incoming traffic allowed to access the database server. In the example, we allowed MySQL/Aurora traffic (port 3306) from the "public-instance-sg" to permit the EC2 instance to communicate with the database.

 **Note:** Correctly configuring security groups ensures proper network access between resources and enhances security.

 **Learn More:** Explore AWS documentation on [Security Groups](#) for detailed information.

## STEP 7: Configure the WordPress installation and connect it to a database

To configure the WordPress installation and connect it to a database, follow these steps(*Figure 2.22: DB Connection page*);

1. Set MySQL Environment Variable:

Replace `<your-endpoint>` with your RDS endpoint. Run the following command in your terminal.

 **Command:**

```
export MYSQL_HOST=<your-endpoint>
```

 **Note:** This command sets the MySQL environment variable to your RDS endpoint.

2. Connect to the WordPress Database:

Replace `<your-username>` and `<your-password>` with your MySQL username and password. Use the following command to connect to the WordPress database:

 **Command:**

```
mysql --user=<your-username> --password=<your-password>
wordpress
```

 **Note:** Connects you to the WordPress database using your MySQL credentials.

3. Create New Database and Grant Permissions:

Inside the MySQL prompt, create a new DB for WordPress and grant permissions:

 **Command:**

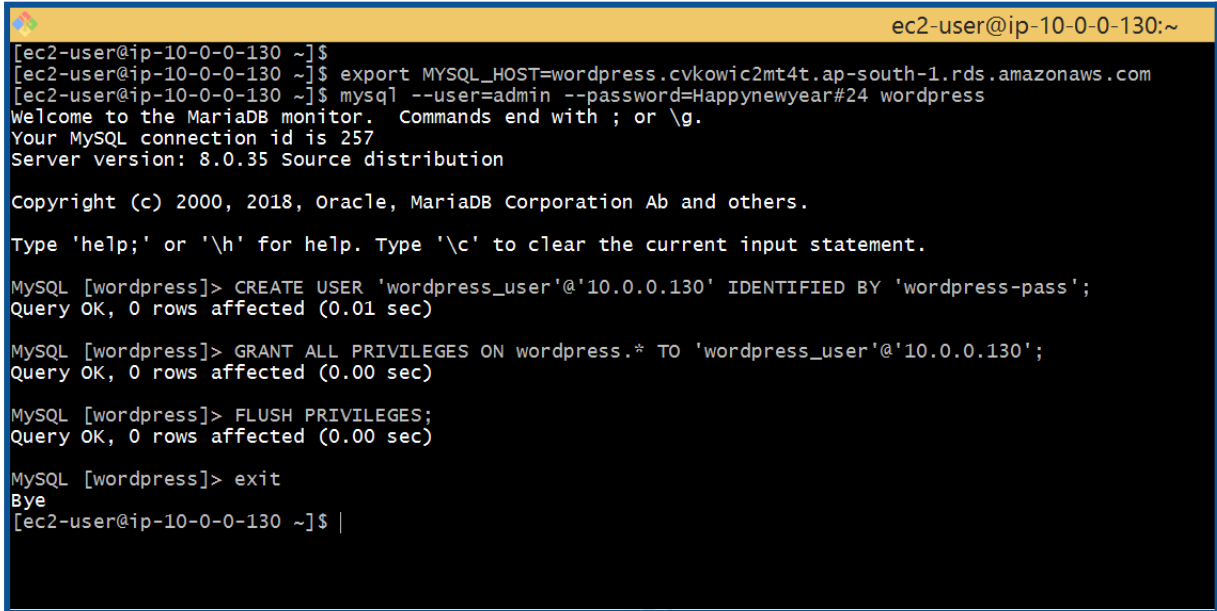
```
CREATE USER 'wordpress'@'localhost' IDENTIFIED BY
'wordpress-pass';
```

```
GRANT ALL PRIVILEGES ON wordpress.* TO wordpress;
FLUSH PRIVILEGES;
```

 **Note:** Creates a new database and grants necessary permissions to the WordPress user.

4. Exit MySQL Prompt:  
Exit the MySQL prompt:

 **Command:** `EXIT;`



```
ec2-user@ip-10-0-0-130:~
[ec2-user@ip-10-0-0-130 ~]$
[ec2-user@ip-10-0-0-130 ~]$ export MYSQL_HOST=wordpress.cvkowic2mt4t.ap-south-1.rds.amazonaws.com
[ec2-user@ip-10-0-0-130 ~]$ mysql --user=admin --password=Happynewyear#24 wordpress
Welcome to the MariaDB monitor. Commands end with ; or \g.
Your MySQL connection id is 257
Server version: 8.0.35 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [wordpress]> CREATE USER 'wordpress_user'@'10.0.0.130' IDENTIFIED BY 'wordpress-pass';
Query OK, 0 rows affected (0.01 sec)

MySQL [wordpress]> GRANT ALL PRIVILEGES ON wordpress.* TO 'wordpress_user'@'10.0.0.130';
Query OK, 0 rows affected (0.00 sec)

MySQL [wordpress]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)

MySQL [wordpress]> exit
Bye
[ec2-user@ip-10-0-0-130 ~]$ |
```

Figure 2.22: DB Connection page

5. Access the wp-config.php file: Use the nano text editor to open the wp-config.php file(Figure 2.23: Nano text editor)

 **Command:**

`nano wp-config.php`

```
ec2-user@ip-10-0-0-130:/var/www/html/wordpress
ut@tenobo MINGW64 ~/downloads
$ ssh -i "Keypair_EpicReads.pem" ec2-user@65.0.204.207
Last login: Mon Dec 25 16:28:14 2023 from 49.47.71.151

#
Amazon Linux 2
###
AL2 End of Life is 2025-06-30.
###
#/_
V/_
A newer version of Amazon Linux is available!
Amazon Linux 2023, GA and supported until 2028-03-15.
https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-0-0-130 ~]$ cd /var/www/html/
[ec2-user@ip-10-0-0-130 html]$ ls
latest.tar.gz wordpress
[ec2-user@ip-10-0-0-130 html]$ cd wordpress
[ec2-user@ip-10-0-0-130 wordpress]$ ls
index.php wp-activate.php wp-comments-post.php wp-content wp-links-opml.php wp-mail.php wp-trackback.php
license.txt wp-admin wp-config.php wp-cron.php wp-load.php wp-settings.php xmlrpc.php
readme.html wp-blog-header.php wp-config-sample.php wp-includes wp-login.php wp-signup.php
[ec2-user@ip-10-0-0-130 wordpress]$ nano wp-config.php
```

Figure 2.23: Nano text editor

6. Locate and Modify the Database Settings(Figure 2.24: Nano text editor page):

Inside the wp-config.php file, find the section with MySQL settings.

Replace the following placeholders:

- `database_name_here` with the actual name of your WordPress database ('wordpress' in your case).
- `username_here` with the MySQL database username ('wordpress' in your case).
- `password_here` with the MySQL database password ('wordpress-pass' in your case).
- Replace `localhost` with your `RDS endpoint`.

Here is an example of how it might look after modification:

```
ec2-user@ip-10-0-0-130:/var/www/html
GNU nano 2.9.8 wp-config.php

<?php
/**
 * The base configuration for WordPress
 *
 * The wp-config.php creation script uses this file during the installation.
 * You don't have to use the web site, you can copy this file to "wp-config.php"
 * and fill in the values.
 *
 * This file contains the following configurations:
 *
 * * Database settings
 * * Secret keys
 * * Database table prefix
 * * ABSPATH
 *
 * @link https://wordpress.org/documentation/article/editing-wp-config-php/
 *
 * @package WordPress
 */

/** Database settings - You can get this info from your web host */
/** The name of the database for WordPress */
define('DB_NAME', 'wordpress');

/** Database username */
define('DB_USER', 'admin');

/** Database password */
define('DB_PASSWORD', 'Happynewyear#24');

/** Database hostname */
define('DB_HOST', 'wordpress.cvkowic2mt4t.ap-south-1.rds.amazonaws.com');

/** Database charset to use in creating database tables. */
define('DB_CHARSET', 'utf8');

/** The database collate type. Don't change this if in doubt. */
define('DB_COLLATE', '');

/**#@+
 * Authentication unique keys and salts.
 *
 * Change these to different unique phrases! You can generate these using
 * the {@link https://api.wordpress.org/secret-key/1.1/salt/ WordPress.org secret-key service}.
 */

^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos M-U Undo
^X Exit ^R Read File ^_ Replace ^U Uncut Text ^T To Spell ^_ Go To Line M-E Redo
```

Figure 2.24: Nano text editor page

 **Note:** Modify database settings in the `wp-config.php` file. Replace placeholders with actual database information.

7. Navigate to the WordPress Secret Key Generator available at <https://api.wordpress.org/secret-key/1.1/salt/> and copy the content generated on the page. Replace the current script labeled as `/*#@+` in your configuration file with the newly acquired values (Figure 2.25: Nano text editor secret key page)

```

ec2-user@ip-10-0-0-130:/var/www/html
GNU nano 2.9.8 wp-config.php

define('DB_NAME', 'wordpress');

/** Database username */
define('DB_USER', 'admin');

/** Database password */
define('DB_PASSWORD', 'Happynewyear#24');

/** Database hostname */
define('DB_HOST', 'wordpress.cvkowic2mt4t.ap-south-1.rds.amazonaws.com');

/** Database charset to use in creating database tables. */
define('DB_CHARSET', 'utf8');

/** The database collate type. Don't change this if in doubt. */
define('DB_COLLATE', '');

/**#@+
 * Authentication unique keys and salts.
 *
 * Change these to different unique phrases! You can generate these using
 * the {@link https://api.wordpress.org/secret-key/1.1/salt/ WordPress.org secret-key service}.
 *
 * You can change these at any point in time to invalidate all existing cookies.
 * This will force all users to have to log in again.
 *
 * @since 2.6.0
 */
define('AUTH_KEY', 'adi2Kk=c^+8LT03t9;ilm|fb*5FR@ebzm#$^~#N|6R|;!0`cx*&Rux|fG}0;U+w');
define('SECURE_AUTH_KEY', 'U-e+1)sN-tQ~E?T~`R1Q|R(SHpffj|2/T~;EVj-P=j^Rwm0 }GI+|cZHMV|0a_KP');
define('LOGGED_IN_KEY', 'ZX2?{7@4zSS~7ZI6_J*kThOZxDt6^&Jk{y>~-*A2|94& GedU0kG4nuBu#LhZdL');
define('NONCE_KEY', 'e2)=-Jp!,5X!52Z1 .|6Hh/jTa*9LM0_GD1-1+F (I6hP1CjO)~&V<xHR&g<n9+Z');
define('AUTH_SALT', 'l[4B`&S1j=V;Q#0iU_SB-%: `bgwr5B^=Eh;-Y{G-b,8nAK9LW95FzKr.XZj8qnM');
define('SECURE_AUTH_SALT', 'BHpII e}c8x7y]b@%+%^`^q+ 8N&BqusBp?-QxMyw<t4b-XS,Z+.|M(-sh<e_;TD');
define('LOGGED_IN_SALT', 'B|gN.^u-AG<X-c-JQQ8yB8Ma sV+[z|!6+4;VNCz0;?,x..Y0!A_--K+{i2TSVz5');
define('NONCE_SALT', 'J9UXJ|G `;eYwXlhs67h.|?[[]0X2^GJ4pkiSg1>KH4[6jHhjJF#k?26-$?iash;');

/**#@-*/

/**
 * WordPress database table prefix.
 *
 * You can have multiple installations in one database if you give each
 * a unique prefix. Only numbers, letters, and underscores please!
 */

```

Figure 2.25: Nano text editor secret key page

8. To allow 'W3TC' plugin write the configuration data into DB

Paste the below command at the end of the script (*Figure 2.26: 'W3TC' plugin config* )

#### Command:

```
define('W3TC_CONFIG_DATABASE', true);
```

```

GNU nano 2.9.8 wp-config.php

*
* @link https://wordpress.org/documentation/article/debugging-in-wordpress/
*/
define('WP_DEBUG', false);

/* Add any custom values between this line and the "stop editing" line. */

/* That's all, stop editing! Happy publishing. */

/** Absolute path to the WordPress directory. */
if (! defined('ABSPATH')) {
 define('ABSPATH', __DIR__ . '/');
}

/** Sets up WordPress vars and included files. */
require_once ABSPATH . 'wp-settings.php';
define('W3TC_CONFIG_DATABASE', true);

```

Figure 2.26: 'W3TC' plugin config

#### 9. Save the Changes:

After making the modifications, press `Ctrl + O` to save the file.

Press `Enter` to confirm the file name.

To exit nano, press `Ctrl + X`.

#### 10. Run commands to deploy Wordpress on your computer

```

sudo amazon-linux-extras install -y
lamp-mariadb10.2-php7.2 php7.2

```

```

sudo yum install php-xml

```

```

cd /home/ec2-user

```

```

sudo cp -r wordpress/* /var/www/html

```

```

Sudo chown -R apache:apache /var/www/html

```

Finally, start hosting the Apache web server

```

sudo systemctl start httpd.service

```

```

Sudo systemctl restart php-fpm

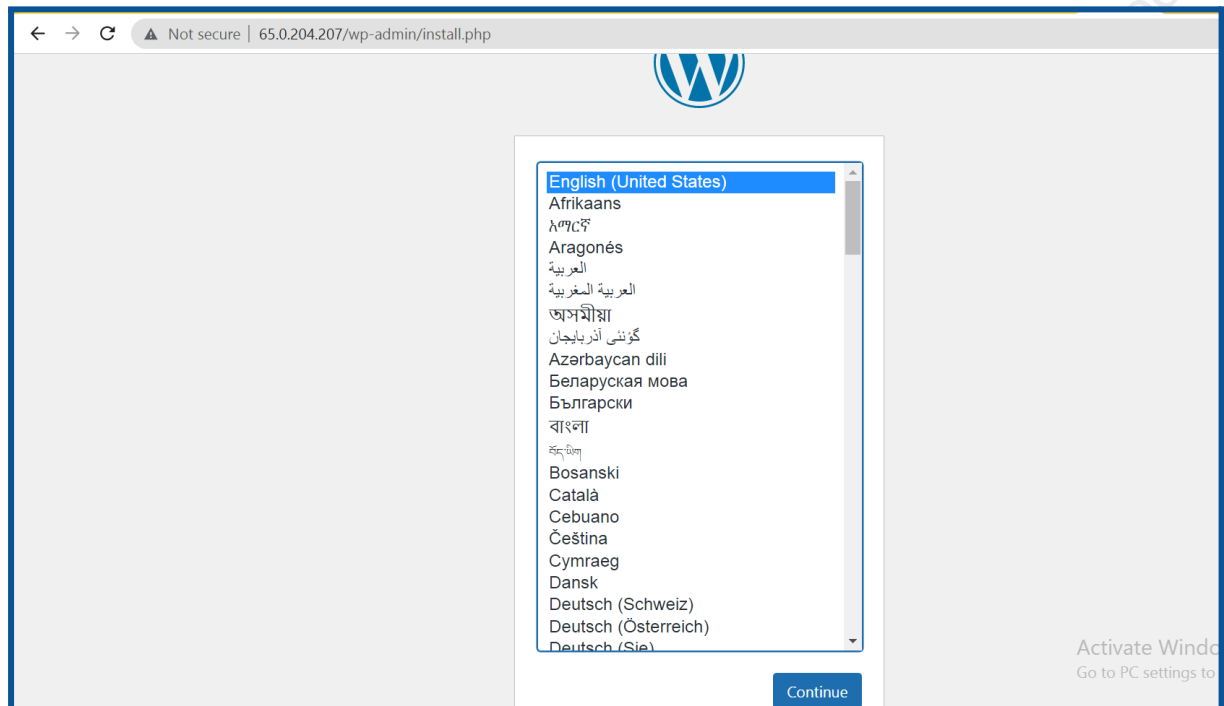
```

## 11. Install WordPress through the Browser:

Open a web browser and enter your EC2 instance's public IP address or domain name(*Figure 2.27: Wordpress language page*).

Follow the WordPress setup wizard:

- a. Select the language and provide database connection details (database name, username, password, and host).



*Figure 2.27: Wordpress language page*

- b. Complete the WordPress setup by providing site details (site title, admin username, password, email, etc.)(*Figure 2.28: Wordpress site details*).

← → ↻ ⚠ Not secure | 65.0.204.207/wp-admin/install.php?step=1

## Welcome

Welcome to the famous five-minute WordPress installation process! Just fill in the information below and you'll be on your way to using the most extendable and powerful personal publishing platform in the world.

### Information needed

Please provide the following information. Do not worry, you can always change these settings later.

**Site Title**

**Username**   
Usernames can have only alphanumeric characters, spaces, underscores, hyphens, periods, and the @ symbol.

**Password**  [Hide](#)  
**Strong**

**Important:** You will need this password to log in. Please store it in a secure location.

**Your Email**   
Double-check your email address before continuing.

**Search engine visibility** ☐ Discourage search engines from indexing this site  
It is up to search engines to honor this request.

[Install WordPress](#)

Figure 2.28: Wordpress site details

## Success!

WordPress has been installed. Thank you, and enjoy!

**Username** admin

**Password** Your chosen password.

[Log In](#)

Figure 2.29: Wordpress Success Page

- c. Login to the WordPress admin dashboard (Figure 2.30: WordPress admin dashboard)

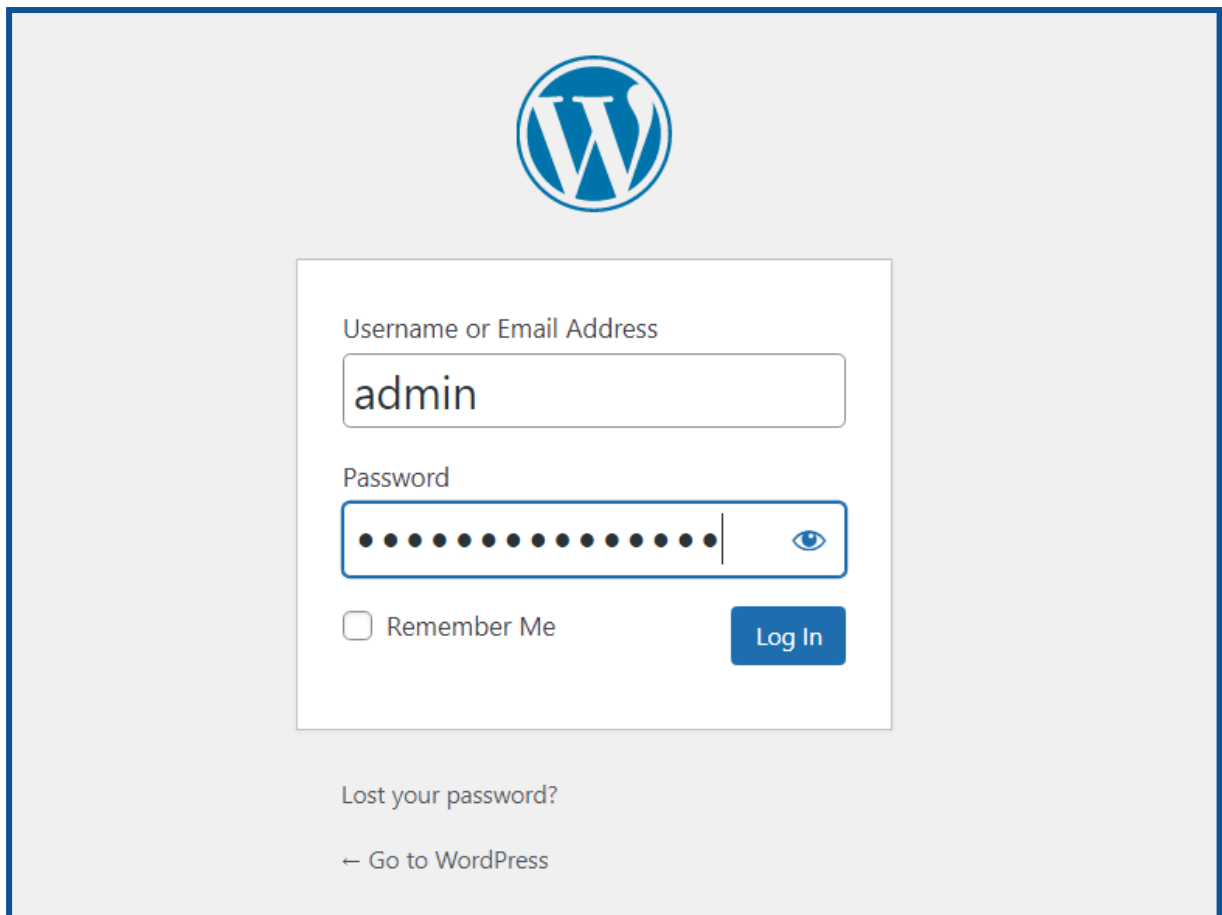


Figure 2.31: WordPress admin dashboard

"**Great job!** You're almost there! This is the final step where you'll access the WordPress app's main page. You've successfully launched, set up, and installed WordPress on your Linux EC2 Instance at EpicReads. Let's take the last exciting step together!" (Figure 2.32: WordPress Welcome Page)

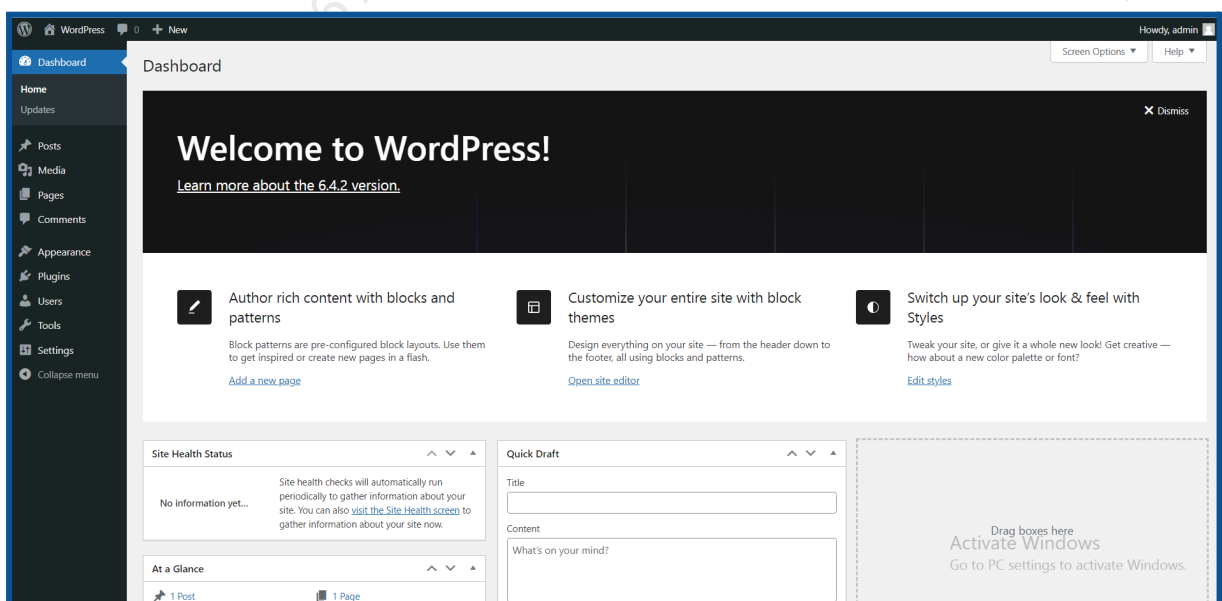


Figure 2.32: WordPress Welcome Page

12. Ensure that the WordPress installation is functioning correctly by creating a test post or page(*Figure 2.33: Add New Post Page*).

Steps to Create a Test Post/Page:

- a. **Navigate to Posts or Pages:** In the WordPress Dashboard sidebar, click on "Add New Post" to create a test post.

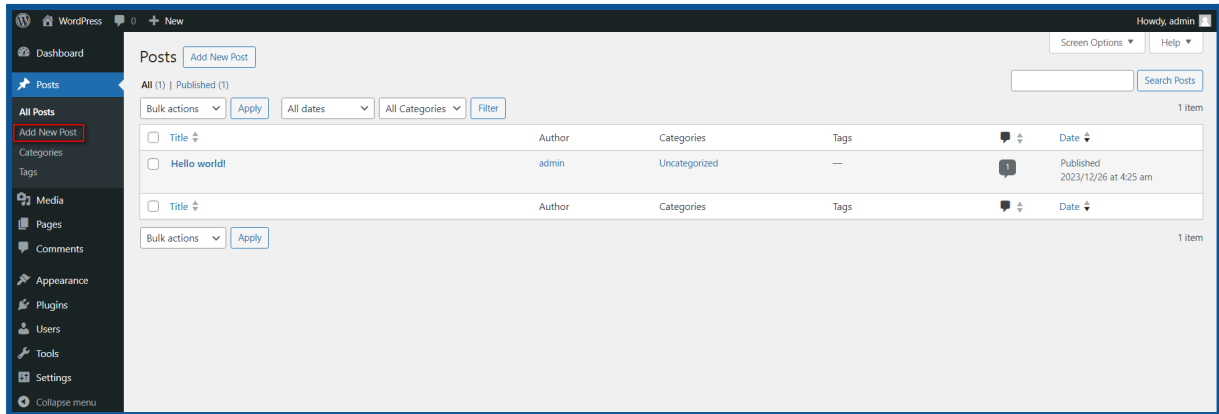


Figure 2.33: Add New Post Page

- b. **Fill in Test Post/Page Details:** Enter a title for the test post or page in the designated field(*Figure 2.34: Post creation Page*).

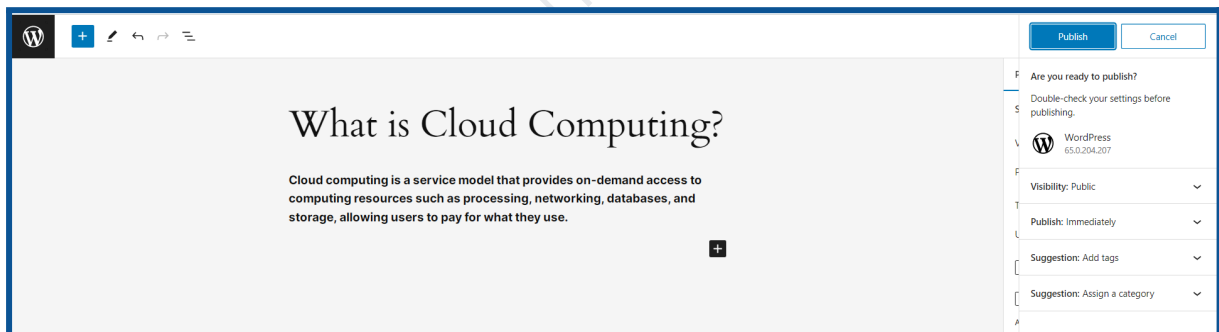


Figure 2.34: Post creation Page

- c. **Publish Test Post/Page:** Once satisfied with the test content, click the "Publish" button to make the post/page live on the website.
- d. **Verify the Test Post/Page:** Visit the live website and navigate to the newly created test post or page(*Figure 2.35: Post Sample Page*)



Figure 2.35: Post Sample Page

 **Note:** Confirm successful WordPress deployment and publication of a test blog on your EC2 Instance at EpicReads.

**Kudos!** On successfully publishing your latest blog via WordPress on your Linux EC2 Instance at EpicReads!

## STEP 8: Domain Configuration (Optional):

This step is the optional step, if you want to configure a domain name to point to the EC2 instance, providing a user-friendly way to access the WordPress site.

1. **Purchase a Domain Name:** Register a domain name through a domain registrar like **GoDaddy, Namecheap, Google Domains** etc.
2. **Access Domain Registrar's DNS Settings:** Log in to your domain registrar's website and find the DNS (Domain Name System) settings or Domain Management section.
3. **Obtain Public IP of the EC2 Instance:** Go to the AWS Management Console, navigate to EC2, and find your EC2 instance's public IP address or public DNS.
4. **Allocate Elastic IP (Optional, but Recommended):** For a more stable IP address, allocate an Elastic IP address within the AWS EC2 console and associate it with your EC2 instance.
5. **Update DNS Records:** In your domain registrar's DNS settings, add an 'A record' or an 'Alias record' (if supported) that points to the Elastic IP address or the public IP of your EC2 instance.

- For 'A record': Enter the hostname (e.g., www) and the IP address of your EC2 instance.
- For 'Alias record' (if supported): Select the AWS service (like EC2) and choose the instance you want to associate with the domain.
- 6. **Configure WordPress:** Once the DNS settings propagate (which might take up to 48 hours), update the WordPress settings to reflect the new domain.
- Log in to the WordPress admin dashboard.
- Go to Settings > General and update the WordPress Address (URL) and Site Address (URL) to the new domain.
- 7. **Test Your Domain:** Open a web browser and enter your domain name (e.g., [www.yourdomain.com](http://www.yourdomain.com)). It should now load the WordPress site hosted on your EC2 instance.

 **Learn More:** Explore further details on domain configuration using [AWS Managed Domain](#).

## STEP 9: Security Enhancements:

Enhance security measures for the WordPress installation by implementing various methods, including configuring the .htaccess file, establishing correct file permissions, and integrating essential security plugins.

1. **Keep WordPress Updated:** Ensure your WordPress core, themes, and plugins are always up-to-date. Regularly check for updates and apply them promptly to patch any security vulnerabilities.
2. **Use Strong Credentials:** Avoid using default usernames like "admin" and use complex passwords that include a combination of letters, numbers, and special characters. Consider using a password manager to generate and store secure passwords.
3. **Limit Login Attempts:** Implement a plugin (e.g., Limit Login Attempts Reloaded) to restrict the number of login attempts. This prevents brute-force attacks by locking out users or bots after multiple failed login attempts.
4. **Enable Two-Factor Authentication (2FA):** Use a plugin (such as Google Authenticator or Wordfence) to add an extra layer of security by requiring a second form of verification, typically a temporary code sent to your mobile device.

5. **Protect wp-config.php:** Add the following code to your .htaccess file to restrict access to wp-config.php:

```
<files wp-config.php>
order allow,deny
deny from all
</files>
```

6. **Secure .htaccess File:** Protect your .htaccess file by ensuring it has the correct permissions (e.g., 644) to prevent unauthorized access. Additionally, regularly check for any unusual modifications.
7. **File Permissions:** Set appropriate file permissions for WordPress files and folders:
- **Directories: 755**
  - **Files: 644**
  - **wp-config.php: 600**
8. **Install Security Plugins:** Consider using security plugins such as Wordfence Security, Sucuri Security, or iThemes Security. These plugins offer features like firewall protection, malware scanning, and real-time threat detection.
9. **Enable HTTPS:** Encrypt data transmitted between users and your website by installing an SSL certificate. This can be done through your hosting provider or using services like Let's Encrypt. Update your WordPress settings to use HTTPS.
10. **Disable Directory Listing:** Prevent directory browsing by adding this line to your .htaccess file:
- ```
Options -Indexes
```
11. **Regular Backups:** Set up regular backups of your WordPress site (files and database) using plugins or your hosting provider's backup solutions. This ensures you can restore your site if needed.